The National University of Computer and Emerging Sciences, Lahore

Course Notes

Compiler Construction

CS-4031

Semester Fall-2026

Compiled lecture notes – see individual lectures for per-lecture references.

Contents

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 1

How Programs Run: Compilers, Interpreters, and JIT Compilation (a Java Case Study); the Language-Processing System; Why Study Compilers; Kinds of Compilers

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.1 (Language Processors), §1.2 (Language-Processing System – previewed here, detailed in Lecture 2), §1.4 (Compiler/Interpreter Distinction), §1.5 (Applications of Compiler Technology)
Other references:	[ET] Engineering a Compiler §1.1; [AP] Modern Compiler Implementation, Ch. 1

Here is what today actually covers:

Lecture Roadmap

1. What is a language processor, and what exactly is a *compiler*?
2. Compilers vs. interpreters vs. just-in-time (JIT) compilation — three ways to run a program, with a full case study of how **Java** uses this to become portable.
3. A **typical language-processing system**: preprocessor, compiler, assembler, linker, and loader – and exactly what an **object file** is.
4. Why study compilers at all, even if you never write one professionally?
5. Applications of compiler technology far beyond "turning C into an executable."
6. Six different **kinds of compiler** you'll encounter by name in industry.

This lecture is entirely conceptual — no algorithms yet. Lecture 2 goes much deeper into the internal *structure* (phases) of a compiler.

1.1 What Is a Language Processor?

[DB] A **language processor** is any program that takes another program written in some source notation and does something useful with it — runs it, translates it, checks it, or transforms it. Compilers are the most famous member of this family, but not the only one.

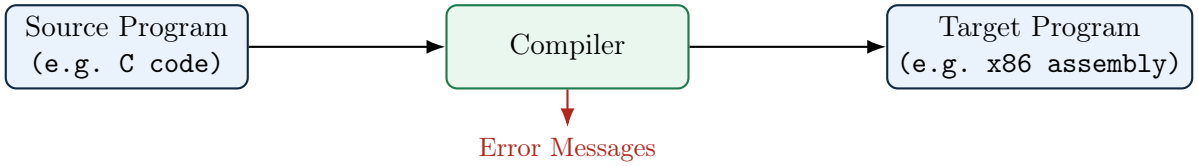
Definition: Compiler

A **compiler** is a program that reads a program written in one language — the *source language* — and translates it into an equivalent program in another language — the *target language*. Along the way, if the source program has errors, the compiler should report them to the user in a useful way.

Three things are packed into that definition and worth pulling apart, because exam questions love to test exactly this distinction:

- **Translation, not execution.** A compiler's job is to *produce* a target program. It does not itself run your code. If you compile `main.c` with GCC, GCC's job ends when it hands you `a.out`; a completely different program (the operating system's loader, then the CPU) is what actually executes it.

- **Equivalence.** The target program must have the same meaning as the source program. This is the single hardest engineering promise a compiler makes, and essentially the entire field of compiler optimization exists to preserve this promise while still making the target program faster.
- **The target language need not be machine code.** Nothing in the definition says what the target language *is* – only that it is "another language." Machine code is the single most common and most visible target, which is why people default to thinking "compiler = produces machine code," but it is only one case. A compiler whose target is *another high-level language* is a source-to-source compiler (§1.6); a compiler whose target is bytecode for a virtual machine (`javac` → JVM bytecode, §1.2.5) is still a compiler by this same definition, even though no physical CPU can execute its output directly.



Exam Focus

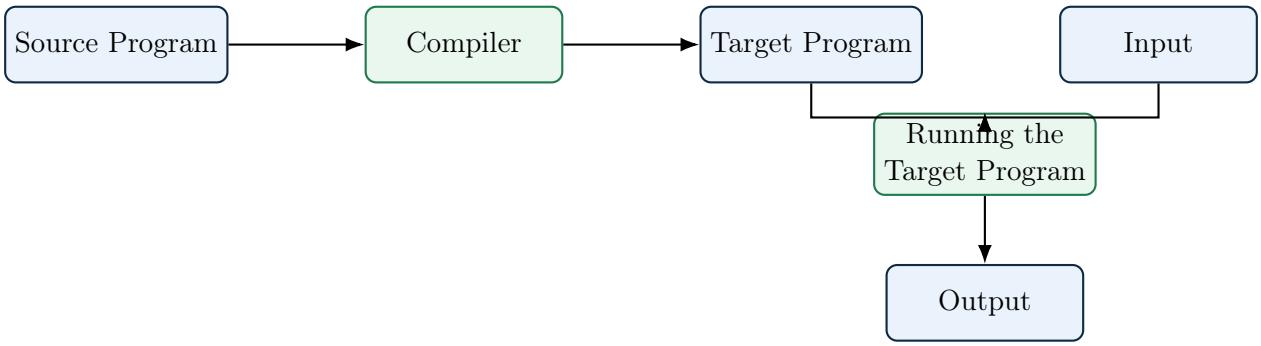
Be able to state, in one sentence, that a compiler *translates*; it does not *execute*. A one-line definition question like "define a compiler" is a classic first-quiz question and students routinely lose marks by describing what an interpreter does instead – or by adding "... into machine code" to the definition, which is not required and is contradicted by transpilers and by `javac` (§1.6).

1.2 Compilers vs. Interpreters vs. JIT Compilation

This is the heart of today's lecture (DB §1.4). There are three broad strategies for getting from source code to a running program, and almost every real language-processing system you use day to day is one of these three, or (very commonly) a hybrid.

1.2.1 Pure Compilation (Ahead-of-Time)

[DB] The compiler translates the entire source program into a target program *before* execution begins, and the target program can then be run any number of times, independently of the compiler. In the classic AOT case that gives this strategy its name, the target language is machine code for the host machine – this is what C, C++, Rust, and (classic) Fortran compilers produce – but the "ahead-of-time, whole-program-at-once" strategy itself is independent of that choice of target (a transpiler, §1.6, is also AOT; it just targets another high-level language instead of machine code).



Advantage – maximal optimization opportunity. Because the entire source program is available before a single instruction runs, an AOT compiler can afford analyses and transformations that a system under run-time time pressure cannot: multiple, deliberately ordered optimization

passes; *whole-program* (interprocedural) analysis such as global register allocation or cross-function inlining; and expensive, exhaustive search over code-generation choices. None of this has to finish before the user sees output, since compilation happens entirely offline – only the resulting target program's run time is on the user's clock.

Disadvantage – the output is not portable. The target program an AOT compiler produces is machine code for one specific instruction set, calling convention, and (usually) operating system. A binary built for x86-64 Linux will not run on ARM64 macOS: there is no platform-independent artifact to ship. Reaching a new target means recompiling from source for that target (or cross-compiling, §1.6) – unlike an interpreter, which ships the same source to every platform, or a bytecode/JIT system, which ships the same portable bytecode everywhere.

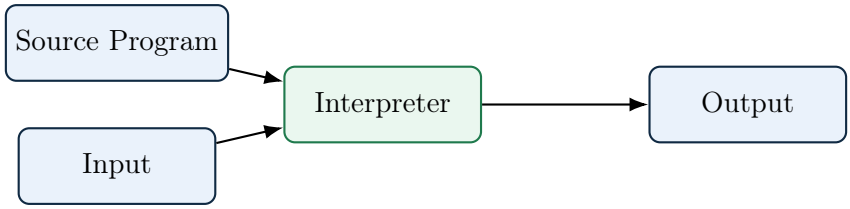
Trade-off, restated: compilation itself can be slow and happens "offline," but every run of the target program afterward is fast, because all the translation work – and all the optimization it enables – has already been paid for once, at the cost of tying the result to one platform.

1.2.2 Pure Interpretation

[DB] An **interpreter** does not produce a separate target program at all. Instead, it directly executes the operations specified in the source program, using the source program (or a close representation of it) as input on every single run.

Definition: Interpreter

An interpreter is a language processor that reads a source program and, using the input supplied by the user, *directly* produces the output of that program — with no separate translation phase that produces standalone target code the user can run independently.



Classic (older) Python, shell scripts, and simple Basic interpreters are close to this pure model: the interpreter re-examines and re-dispatches on the source (or its parsed form) every single time a statement executes, even inside a loop that runs a million times.

Advantage – portability. The same source program (or bytecode) runs unmodified on any machine that has a compatible interpreter, with no separate build step per target platform. This is the mirror image of the AOT compiler's disadvantage above.

Disadvantage – repeated-work overhead. There is no separate compile step, so you can start running – and get feedback on – code instantly, which is wonderful for interactive development and REPLs. The cost is that the interpreter re-examines and re-dispatches on the source (or its parsed form) on *every* execution of a line, with no persistent, whole-program analysis carried over between runs – the opposite of an AOT compiler's one-time, multi-pass optimization. DB notes this repeated-work overhead can make interpretation **10–100× slower** than running compiled code.

Exam Focus

Know this speed trade-off in both directions: compilation trades slower "build time" for faster "run time"; interpretation trades away build time for slower, but more flexible and immediate, run time. Two further pairings are just as commonly tested: *portability* is a classic interpreter advantage over AOT compilation (the same source runs unmodified on any machine that has the interpreter, whereas an AOT binary is tied to one target and needs a recompile elsewhere), while *optimization opportunity* runs the other way – an AOT compiler sees the whole program before

run time and can afford multiple, expensive, whole-program passes, while a pure interpreter re-examines the source on every execution and so cannot afford to.

1.2.3 The Hybrid Reality: Bytecode + JIT

[beyond DB] In practice, almost no serious modern language is *purely* compiled or *purely* interpreted. DB §1.4 briefly mentions Java's model as a hybrid; it is worth understanding this hybrid model properly, because it is how the overwhelming majority of code you run today — Java, C#, JavaScript, modern Python (via PyPy), and even Python's own CPython — actually executes.

Definition: Virtual Machine (VM), vs. a Plain Interpreter

A **virtual machine**, in the sense used here, is a program that simulates a computer: it defines its own instruction set (an abstract, software-only ISA) together with the state a real CPU would have – a program counter, and either an operand stack (JVM, Wasm) or a set of virtual registers – and then runs a fetch-decode-execute loop over that instruction set, exactly the way real hardware executes machine code, except every step is simulated in software. Crucially, a VM does *not* interpret your source program directly. There is always a separate front-end compilation step first (described just below) that translates source into **bytecode** – instructions for this abstract machine – and the VM interprets *that*, not your original code.
This is different from a "plain" (tree-walking) interpreter, of the kind §1.2.2 first introduced: a tree-walking interpreter parses source into an AST and directly, recursively evaluates that tree (`evaluate(node)` calling `evaluate(node.left)`, and so on). There is no separate instruction set, no fixed opcode encoding, and no simulated CPU state – the evaluator's own call stack mirrors the shape of the *source program*, not the shape of a machine. Nothing about it resembles hardware, so it does not earn the name "virtual machine."
The distinction, side by side:

- **What is executed.** VM: a fixed, formally-defined bytecode instruction set, produced by a separate compile step. Tree-walking interpreter: the source program's own parse tree, walked directly, with no separate instruction set at all.
- **Execution state.** VM: an explicit program counter plus an operand stack or virtual registers – state modeled on real hardware. Tree-walker: just the recursive call stack of the `evaluate` function, shaped like the program's grammar, not like a CPU.
- **Is it a genuine compilation target?** VM: yes – bytecode is often documented and shared across multiple front-end languages (JVM bytecode is targeted by Java, Kotlin, Scala, and Clojure alike). Tree-walker: no – the AST is just this one interpreter's private internal representation of this one program.

Every VM is (or contains) some form of interpreter – it interprets bytecode instruction by instruction. But not every interpreter is a VM: a tree-walker interprets source structure directly and never defines a machine to simulate. (A second, unrelated sense of "virtual machine" – *system* VMs like VMware or QEMU, which virtualize an entire computer including its OS – is a different concept entirely and not what is meant here.)

1. **Source → bytecode.** A "front-end" compiler translates source code into a compact, machine-independent intermediate form called **bytecode** (Java's `.class` files hold JVM bytecode; Python's `.pyc` files hold Python bytecode).
2. **Bytecode → execution.** A **virtual machine (VM)** then runs this bytecode. It can do this two ways:
   - *Interpret* the bytecode directly, instruction by instruction (slow but starts instantly) – this is what most bytecode VMs do at first.
   - *Just-in-time (JIT) compile* hot pieces of bytecode — the loops and functions that run

over and over — into native machine code *while the program is running*, and cache that native code for reuse.

Definition: Just-In-Time (JIT) Compilation

JIT compilation defers translation to native machine code until run time, and does so selectively: only code that is actually hot (executed frequently) is compiled, while cold code may remain interpreted. This lets a system get the fast startup of an interpreter and the fast steady-state speed of a compiler.

Why "Warm-Up" Adds a Delay

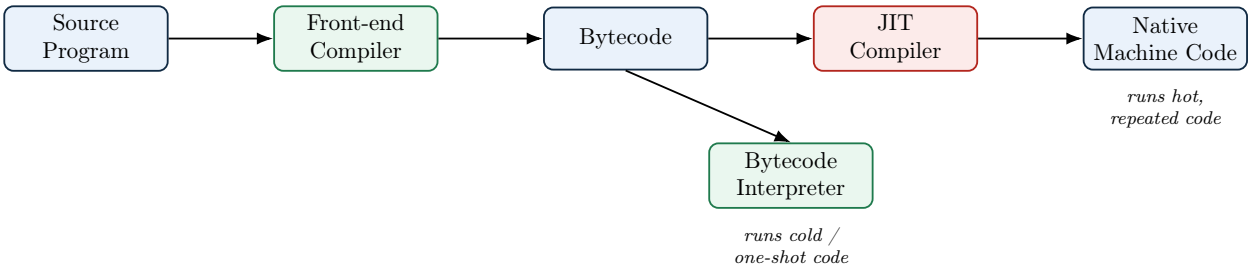
The "time to reach peak speed" row in §1.2.4's table (distinct from startup latency – see the exam-focus note there) has a concrete cause: three costs stack up, in sequence, before any piece of code reaches its fastest version, and none of them can be skipped.

1. **Early runs pay interpreter-speed cost.** The very first time a method or loop executes, no compiled native code for it exists yet – there is nothing to jump to – so it runs through the interpreter (or a cheap baseline tier, e.g. HotSpot's C1), which is inherently slower per operation than optimized native code.
2. **The system must first observe that code is hot.** A JIT does not compile on the first call; it counts. The runtime increments an invocation/back-edge counter on every call or loop iteration and only triggers compilation once that counter crosses a threshold. A loop that only runs a handful of times may never get compiled at all, because counting to that threshold, by construction, requires real executions to have already happened at the slower tier.
3. **Compilation itself takes real, non-trivial time.** Once flagged hot, the method still has to go through the JIT's own pipeline – build an IR from the bytecode, run optimization passes (inlining, register allocation, and more), and emit native code. This usually runs on a background thread so it does not block the program outright, but it still competes for CPU and memory (see "Extra runtime overhead" in §1.2.4's table), and the *old*, slower version keeps running until the newly compiled version is ready and swapped in.

The practical consequence: long-running processes (servers, long batch jobs) amortize this one-time warm-up cost over a long steady-state run and benefit enormously from JIT compilation, while short-lived scripts or CLI invocations may finish before ever reaching peak speed – exactly why AOT-vs.-JIT benchmarks can be misleading if they measure only a program that runs for a second or two.

Example: Tiered execution in real systems

- **JVM (HotSpot):** interprets bytecode first; the C1 ("client") JIT compiles warm methods quickly with light optimization; the C2 ("server") JIT recompiles the hottest methods with heavy optimization once enough profiling data exists.
- **V8 (Chrome/Node.js JavaScript engine):** `Ignition` interprets bytecode; `Sparkplug` does a fast baseline JIT; `TurboFan` does an aggressive optimizing JIT for hot functions, and can *deoptimize* back to the interpreter if an assumption used for optimization (e.g. "this variable is always a number") turns out to be wrong.
- **CPython (standard Python):** traditionally a pure bytecode interpreter with no JIT; `PyPy` is an alternative Python implementation that adds a JIT and is often 5–10× faster on long-running code as a result.



Extra Depth: AOT with a JIT Fallback

DB draws the compiler/interpreter line fairly simply. In practice, the line is blurrier still: some modern systems (GraalVM Native Image, .NET's ReadyToRun) precompile most code AOT for fast startup but still keep a JIT available for code discovered or specialized at run time. So "compiled" and "JIT-compiled" are not always mutually exclusive labels for one system. *(Transpilers, cross-compilers, bootstrapping compilers, and decompilers are their own category of "kind of compiler" rather than a compiler/interpreter distinction – we cover all of these properly in §1.6.)*

1.2.4 Comparing the Three Models

Table 1: Comparing the three execution models

	Pure Compiler	**Pure Interpreter**	**JIT / Hybrid**
When translation happens	Entirely before run time	Continuously, during run time	Source → bytecode happens before run time; bytecode → native code (the JIT step itself) is **always fully during** run time – see note below
Time to reach peak speed	N/A – already at peak from instruction one (see above)	N/A – there is no "peak"; every instruction runs at the same interpreted speed forever	**Nonzero ("warm-up")** – profiling must first identify hot code, then compilation must finish, before that code is upgraded to native speed; the program is already running the whole time this happens
Steady-state speed	Fastest	Slowest (re-dispatch overhead)	Approaches compiled speed for hot code
Portability	Target is machine-specific – a new platform needs a recompile (or cross-compile)	Source runs anywhere the interpreter exists	Bytecode is portable; the JIT-generated native code is not, but it never has to be shipped
Optimization opportunity	Largest – whole program visible before run time, so multiple, expensive, whole-program passes are affordable	Smallest – source (or bytecode) is re-examined on every execution, so heavyweight analysis is normally too costly to repeat	Bounded by run-time budget – fewer/cheaper passes than AOT and limited mostly to hot methods, but compensated by real profiling data (next row) unavailable to AOT

continued on next page

Table 1 – comparing the three execution models (continued)

	Pure Compiler	**Pure Interpreter**	**JIT / Hybrid**
Can use runtime profiling info?	No – only sees the source, never the running program	N/A – re-examines source every time, no persistent profile	**Yes** – observes actual branch frequencies and value types, enabling speculative optimizations no static compiler can safely make
Extra runtime overhead	None – compiler is long gone by run time	Dispatch overhead only	Profiling counters, background compiler threads, and a generated-code cache all compete with the program for CPU/memory
Debuggability / interactivity	Weakest (must recompile to test a change)	Best (REPL, immediate feedback)	Good (often still supports a REPL over bytecode)
Typical users	C, C++, Rust, Fortran	Shell scripts, classic BASIC	Java, C#, JavaScript, PyPy

Exam Focus

The "When translation happens" row is the row most often misread. The hybrid pipeline has *two* separate translation stages, and only one of them is "just-in-time":

- **Source → bytecode** (e.g. `javac` producing `.class` files) is ordinary ahead-of-time compilation. It happens once, before the program is ever run, exactly like §1.2.1's pure-compiler case – it is just that its *target* language is bytecode instead of machine code.
- **Bytecode → native code**, the actual JIT step, is *never* done ahead of time, not even partially. The decision of *which* loop or method is hot enough to be worth compiling is made by watching the program actually run (via profiling counters), and the native code generated is specific to the behavior observed *during that run* – this is exactly what "just-in-time" means: translation deferred until the last possible moment, triggered by run-time evidence, not decided in advance.

So the "partly before, partly during" language in the table describes the *full hybrid pipeline* (front-end compile, then JIT), not the JIT step in isolation. Taken alone, JIT compilation is entirely a run-time activity, no different in *when* it happens from pure interpretation – what makes it different is *what* it produces (a reusable native-code artifact) rather than *when* it produces it.

Exam Focus

A related trap, worth knowing even though the table above does not track it separately: "startup latency" and "warm-up delay" sound like the same thing but are not. **Startup latency** asks only: how long before the program starts executing at all? By that measure, JIT and pure interpretation are identical – a JIT-based system *begins* by interpreting (or running a cheap baseline tier) exactly like a pure interpreter does, so there is no extra pause before the first instruction runs. "Warm-up," by contrast, is not a delay *before* starting; it is the time *after* the program has already started before some of its code gets promoted to native speed – exactly what the "Time to reach peak speed" row above captures. The program is running the entire time warm-up is happening – it is simply running at interpreted (not yet peak) speed for that stretch. So it is correct to say a JIT system has low startup latency *and* a nonzero warm-up period – these are not in tension, because they answer two different questions.

A Bit of History

Interpretation is not a modern shortcut invented for scripting languages — it is *older* than compilation. Early LISP (1958) was interpreted because writing a LISP compiler was itself a research problem; interpreters let you experiment with a language before anyone knew how to compile it well. JIT compilation is also older than most people assume: the term traces to Smalltalk and self-modifying-code systems of the 1980s, and was popularized for the mainstream by Sun's HotSpot JVM in 1999 – it was Java's answer to "we want C-like speed but Smalltalk-like portability and safety."

1.2.5 Case Study: How Java Achieves Portability

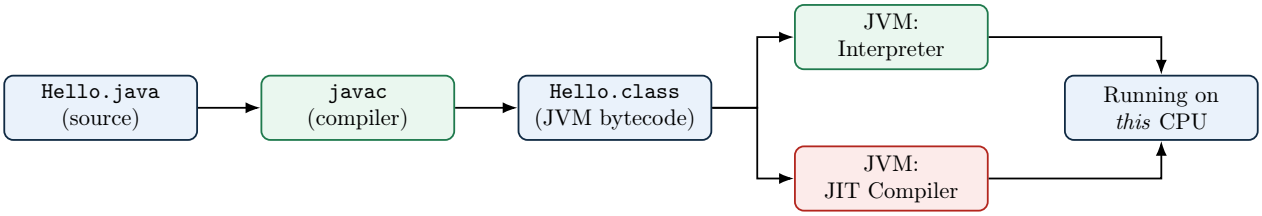
[beyond DB] Java is the single clearest real-world illustration of everything in §1.2, so it is worth walking through as a complete case study — especially since Java's own design goals were *explicitly* about the compiler/interpreter trade-off from the start.

1.2.5.1 From Pure Interpreter to JIT: A Short History

Era	What Happened
Java 1.0 (1996)	The original JVM was a **pure bytecode interpreter**. Every bytecode instruction was fetched, decoded, and dispatched on every single execution – simple and portable, but roughly 10–20× slower than equivalent compiled C, which badly hurt Java's early reputation for performance.
Java 1.2 – "HotSpot" research	Sun's research team built an adaptive compiler codenamed **HotSpot**: instead of compiling everything, or nothing, it would *watch the program run* and compile only the small fraction of methods that actually matter for performance.
Java 1.3 (2000) onward	HotSpot became the default production JVM. This is the point Java became a genuine **hybrid**: start by interpreting (fast startup), then JIT-compile whichever methods turn out to be "hot" (fast steady state). Every mainstream JVM since has kept this same architecture.

1.2.5.2 Compilation to Bytecode, Then Portability via the JVM

[DB] `javac`, the Java *compiler*, does not produce machine code for any real CPU at all. It produces **Java bytecode** – instructions for an abstract, idealized machine called the **Java Virtual Machine (JVM)** that does not physically exist as hardware.



"Write Once, Run Anywhere"

Because `javac` only ever has to target *one* abstract machine (the JVM specification) instead of every physical CPU family, the exact same `Hello.class` file runs unmodified on Windows/x86, Linux/ARM, or a JVM on any other platform — as long as *that* platform has a compliant JVM installed. All the platform-specific work is pushed into the JVM implementation itself (one per platform, built once by the JVM vendor), instead of into every single Java program ever written. This is Sun Microsystems' famous "Write Once, Run Anywhere" pitch for Java, and it is the exact same $M + N$ argument (rather than $M \times N$) you will meet again with LLVM IR in Lecture 2 — just applied to distribution instead of to compiler internals.

1.2.5.3 HotSpot's Tiered JIT in More Depth

[DB] Modern HotSpot uses **tiered compilation**, layering the interpreter and two JIT compilers so the JVM is never stuck doing more work than the code's importance justifies:

1. Every method starts out **interpreted**, while the JVM silently maintains an **invocation counter** (calls) and a **back-edge counter** (loop iterations) for it.
2. Once a counter crosses a threshold, the **C1 (client) compiler** quickly produces a lightly-optimized native version, and also inserts **profiling instrumentation** into it (recording which branches are taken, which concrete types show up at a call site, ...).
3. If the method keeps getting hotter, the **C2 (server) compiler** recompiles it a second time, now using the profiling data gathered by the C1 version to make aggressive, *speculative* optimizations: inlining a virtual method call that has – so far – always resolved to the same concrete type; eliminating a heap allocation entirely if it can prove the object never escapes the method (**escape analysis**); unrolling a hot loop.
4. If a speculative assumption is later violated (a second, different type shows up at that call site), HotSpot **deoptimizes**: it discards the C2 code for that method and falls back to the interpreter (or C1) until it is confident enough to try recompiling again.

Exam Focus

Be able to explain, in your own words, why HotSpot's C2 compiler can safely make optimizations (like assuming a call site's type never changes) that a purely static AOT compiler like GCC never could – and what mechanism (deoptimization) lets it recover if that assumption turns out to be wrong.

1.2.5.4 JIT vs. AOT vs. Interpreter: Advantages and Disadvantages

Pulling together everything from the enhanced table in §1.2.4 and the HotSpot walkthrough above, here is the direct trade-off summary for a JIT compiler specifically:

JIT Advantages	JIT Disadvantages
Can use **real runtime profile data** (actual types, actual branch frequencies) to make optimizations no static compiler can safely attempt – generally impossible for an AOT compiler, which only ever sees the source.	**Warm-up latency**: peak performance is only reached after enough executions to get hot code recompiled – a real problem for short-lived programs (e.g. CLI tools, serverless functions).
Ships one **portable** bytecode artifact instead of a separate binary per target CPU – matching an interpreter's portability, unlike AOT.	Consumes **extra CPU and memory at run time** for profiling counters, the JIT compiler threads themselves, and a generated-code cache – resources an AOT binary or a plain interpreter never spend.
Reaches **steady-state speed close to AOT-compiled native code** for long-running, hot-path-dominated programs – far faster than a pure interpreter ever gets.	**Less predictable** performance moment-to-moment (a method might pause execution while it gets recompiled), which matters for hard real-time systems that AOT compilation suits better.
Can **adapt to the actual host CPU** at run time (e.g. use available vector instructions) without shipping separate binaries per microarchitecture, unlike a single AOT build.	**Significantly more implementation complexity**: an interpreter, multiple JIT tiers, profiling instrumentation, and deoptimization machinery, versus AOT's comparatively simple "compile once, done."

Extra Depth: The Java Class File, Briefly

A `.class` file is itself a small, well-defined binary format worth knowing at a glance: it opens with a **magic number** (`0xCAFEBABE`, a Sun Microsystems in-joke), a class-file version number, a **constant pool** (a table holding every literal, class name, and method/field reference the class uses – similar in spirit to the symbol table we will meet in Lecture 2), and then the actual bytecode for each method. Tools like `javap -c Hello.class` let you disassemble a compiled class and read its raw JVM bytecode instructions directly – a nice hands-on way to see §1.2.3's "token stream → bytecode" pipeline made completely concrete.

1.3 Why Study Compilers? (DB §1.1)

[DB] You may never write a production C compiler. So why is this course core, and why does almost every strong CS program require it? Three overlapping reasons:

1. **Compilers are, in DB's phrase, "one of the largest and most complex software systems" commonly built** – yet a working one is buildable in a single semester project. Nowhere else in the curriculum will you build something this large, this precisely specified, end to end, from a formal problem statement to a working artifact.
2. **The techniques generalize far beyond compilers.** Finite automata power text editors, network protocol parsers, and input validation. Context-free grammars appear in JSON parsers, config-file readers, and query languages. Data-flow analysis underlies static analyzers, IDE "find usages" features, and security vulnerability scanners. You are not just learning to build a compiler; you are learning the standard toolkit for building *any* system that must process structured, language-like input reliably.
3. **Compilers sit at the hardware/software boundary.** Understanding how a `for` loop becomes register moves and branch instructions demystifies performance – why one piece of "equivalent" code runs 10× faster than another, why cache-friendly code matters, and why some

languages can be faster than others for reasons that have nothing to do with the programmer's skill.

Extra Depth: A "Meta" Argument

There's a fourth reason DB does not spell out explicitly but that experienced engineers often cite: compiler construction is one of the few courses where you are forced to build a *multi-stage pipeline* with cleanly separated concerns (lexing, parsing, semantic analysis, code generation) and *formally specified* correctness at each stage. This is directly transferable to designing any large pipeline system – data pipelines, network stacks, even ML training pipelines – where the same discipline of "define an intermediate representation, verify it, transform it, repeat" shows up again and again.

1.4 Applications of Compiler Technology (DB §1.5)

[DB] DB is explicit that compiler techniques reach "far beyond" translating a general-purpose programming language into machine code. Five application areas DB highlights:

1. **Implementation of high-level programming languages.** The obvious case, but note the historical point DB makes: higher-level languages raised the level of abstraction precisely *because* compiler technology matured enough to make that abstraction affordable in performance terms.
2. **Optimizations for computer architectures.** As hardware has grown more parallel (multicore, SIMD/vector units, deep pipelines, GPUs), the compiler has taken on more responsibility for finding parallelism and hiding memory latency that programmers cannot realistically manage by hand.
3. **Design of new computer architectures.** Hardware designers now co-design instruction sets *with* the compiler in mind (RISC philosophy: keep hardware simple, let the compiler do the scheduling), since a feature only helps if a compiler can reliably exploit it.
4. **Program translations.** Beyond source-to-machine-code, DB lists: binary translation (retargeting compiled binaries to a new instruction set, e.g. Apple's Rosetta 2 translating x86 binaries to run on Apple Silicon), hardware synthesis (compiling a hardware description language like Verilog into actual circuit layouts), database query interpreters (translating SQL into an execution plan), and compiled simulation (compiling a simulation model, e.g. for chip verification, into fast executable code rather than interpreting it).
5. **Software productivity tools.** Type checking, bounds checking, and static program-analysis tools that catch bugs before run time all reuse the same front-end machinery (lexers, parsers, semantic analyzers) that compilers use.

Extra Depth: Applications DB Does Not Mention

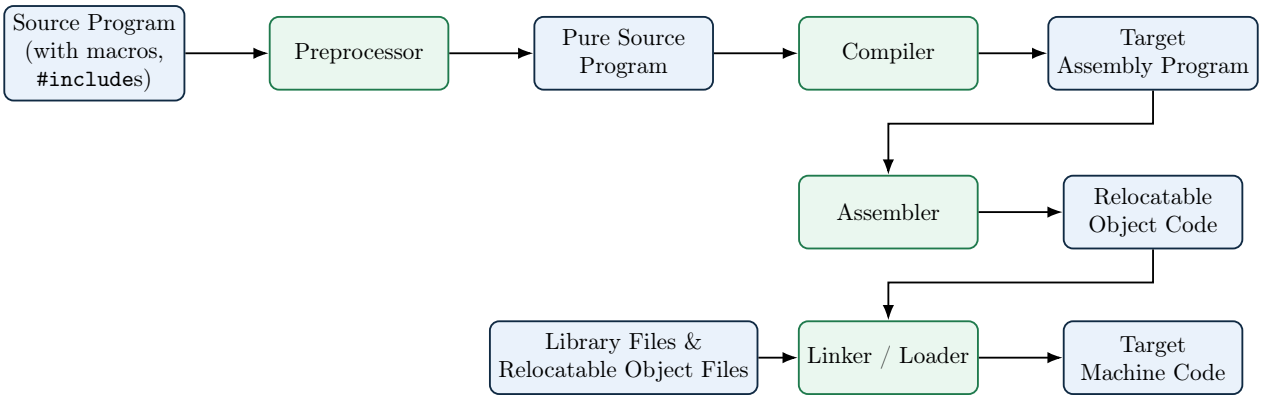
The five areas above were written with early-2000s computing in mind. Several major compiler-technology application areas have grown enormously since and deserve a mention:

- **Machine-learning compilers.** Frameworks like Google's XLA, Apache TVM, and MLIR (Multi-Level Intermediate Representation, itself built using classic compiler theory) compile neural-network computation graphs down to highly optimized GPU/TPU kernels – this is now one of the most active areas of compiler research.
- **WebAssembly (WASM).** A portable, sandboxed bytecode target designed so *any* source language (C, Rust, Go, ...) can compile to it and run safely inside a browser or standalone runtime at near-native speed.

- **Superoptimizers.** Tools (e.g. Souper, STOKE) that search – sometimes with SMT solvers, sometimes with reinforcement learning – for provably optimal short instruction sequences, going beyond what traditional rule-based optimization passes can find.
- **Domain-specific languages (DSLs) and their compilers.** Halide (image-processing pipelines), SQL query planners, regular-expression engines, and shader languages (GLSL/HLSL for GPUs) are all compilers for a language deliberately kept small and specialized so it can be optimized far more aggressively than a general-purpose language could be.
- **Security applications.** Control-flow integrity instrumentation, automatic bounds-check insertion (AddressSanitizer), and fuzzing harness generation are compiler-pass techniques applied specifically to hardening software against attack.
- **LLVM as shared infrastructure.** Perhaps the biggest structural change since DB 2nd edition: LLVM lets dozens of front-ends (Clang/C/C++, Rust, Swift, Julia, and more) share one very well-engineered optimizer and set of back-ends, instead of every language reinventing code generation. We will return to LLVM in Lecture 2.

1.5 A Typical Language-Processing System

[DB] The "compiler" most people invoke by typing `gcc file.c` is not, strictly, one single program at all – it is a **driver** that coordinates several distinct tools in sequence. DB places the compiler proper inside this larger **language-processing system**.



A typical language-processing system (compare DB Fig. 1.5): the "compiler" you invoke on the command line is really a pipeline of five distinct programs.

- **Preprocessor.** Expands macros (`#define`), includes files (`#include`), and resolves conditional-compilation directives (`#ifdef`) — all as pure text substitution, before the compiler's grammar-aware phases ever run.
- **Compiler (proper).** Everything from §1.2: source → target assembly (today's "black box"; Lecture 2 opens it up phase by phase).
- **Assembler.** Translates human-readable assembly instructions into **relocatable machine code** and packages it as an **object file**.
- **Linker.** Combines multiple object files and library files into one program, resolving references between them.
- **Loader.** Places the final program into memory and starts it running.

Is an Assembler a Compiler?

By the definition in §1.1, yes: an assembler reads a program in one language (assembly) and translates it into an equivalent program in another language (relocatable machine code), with no execution of its own along the way – that is exactly the compiler definition. It is given its own name and its own box in the pipeline diagram above for practical, not definitional, reasons: assembly is designed to be an almost line-for-line, one-to-one notation for machine instructions, so an assembler's job skips most of what makes a "full" compiler hard (there is essentially no analysis phase, no optimization search, and little structural gap between source and target to bridge) and reduces mostly to table lookup and address bookkeeping. The lesson generalizes: *compiler* names a *role* – translate one language into another, without executing either – not a claim about how sophisticated the translation has to be. An assembler, a JVM-bytecode compiler like `javac`, and a heavily-optimizing tool like GCC are all compilers in exactly the same technical sense; they differ only in how much analysis and transformation the translation requires.

Definition: Object File

An **object file** is the output of the assembler: a binary file containing machine instructions and data *that is not yet a runnable program.* Critically, it contains:

- **Relocatable code/data**, with memory addresses left as placeholders, since the assembler does not know where in memory this code will ultimately live;
- A table of **symbols this file defines** (e.g. a function it implements, available for other files to call); and
- A table of **symbols this file references but does not define** (e.g. a call to `printf`, defined elsewhere in the C standard library) – these are exactly what the linker must resolve next.

On Linux this is the `.o` file produced by `gcc -c`; the same idea is called an `.obj` file on Windows.

What the Linker Actually Does

Programs are almost never compiled as a single file. The **linker**'s job is to take *several* object files (and pre-built library files) and merge them into one program, by:

1. **Symbol resolution:** matching every symbol a file *references* (like a call to a function defined in another file, or in a library) to the file that actually *defines* it. An unresolved symbol here is exactly the "undefined reference to `foo`" error you see at link time, as distinct from a compiler error.
2. **Relocation:** now that every piece of code has been assigned a real, final address in the combined program, the linker patches every placeholder address left by the assembler with the correct final value.

The output is one combined file — an **executable** — with (in the simplest case) no unresolved symbols and no remaining placeholder addresses.

What the Loader Actually Does

The **loader** is typically part of the operating system, not the compiler toolchain. When you run a program, the loader:

1. Allocates memory for the program (code, data, stack, heap regions);

2. Copies (or memory-maps) the executable's instructions and initial data into that memory;
3. Performs any *final* address fix-ups that were deferred until this exact moment (relevant for dynamically linked libraries, previewed in Lecture 2); and
4. Transfers control to the program's entry point – the actual first instruction that runs, which is *not* always your `main` function directly (a small language-runtime startup routine usually runs first).

Are Addresses Absolute After Linking, or Only After Loading?

It depends on which of two linking models is used, and modern systems mostly use the second one.

- **Fixed-base static linking (the "simplest case" above).** The linker assumes the program will always be loaded at one predetermined virtual address. Given that assumption, the linker already has everything it needs to compute true, final **absolute** addresses during linking itself – the "Relocation" step above is the linker permanently writing those absolute values into the executable. The loader's job then is just to copy bytes into memory at that fixed spot; no address math happens at load time.
- **Position-independent executables (PIE) and shared libraries – the modern default.** A shared library (`.so`/`.dll`) must be loadable at different addresses in different processes, since many unrelated programs share one copy of it; and ASLR (address space layout randomization) deliberately loads a program at a different random base address on every run, as a security defense. In both cases the linker *cannot* know the final load address in advance, so it deliberately leaves references as **relative** offsets (or entries in a relocation table) instead of fixing them. Only at load time, once the loader's dynamic-linking component has actually chosen a base address for that run, do those relative offsets become true run-time absolute addresses – this is exactly the "final address fix-ups" mentioned in step 3 above.

So "linker addresses are relative and become absolute once the loader picks where the program is loaded" is correct specifically for the PIE/shared-library case, which is now the common one – not a universal property of everything the linker produces.

Exam Focus

A very common exam trap: "which phase reports an 'undefined reference' / 'unresolved external symbol' error for a missing function?" The answer is the **linker**, not the compiler — the compiler is perfectly happy as long as a function is *declared* (e.g. via a header file); only the linker checks that a matching *definition* actually exists somewhere among all the object files and libraries it was given.

Extra Depth: Seeing the Real Pipeline on Your Own Machine

You can watch this exact pipeline happen with GCC or Clang by stopping it at each stage:

- `gcc -E prog.c -o prog.i` — run *only* the preprocessor; inspect `prog.i` to see macros expanded and headers inlined.
- `gcc -S prog.i -o prog.s` — run the compiler proper; `prog.s` is human-readable target assembly.

- `gcc -c prog.s -o prog.o` — run the assembler; `prog.o` is a relocatable ELF (on Linux) object file. Inspect it with `objdump -d prog.o` or `nm prog.o`.
- `gcc prog.o -o prog` — run the linker, pulling in the C runtime and any libraries, producing the final executable.

Try this on a one-line `"hello world"` program: the jump in size and content between `prog.i` (hundreds of lines, from expanded standard-library headers) and `prog.c` (one line) is a genuinely eye-opening way to internalize what the preprocessor actually does.

Extra Depth: Static vs. Dynamic Linking, and What the Loader Really Does

The linker/loader description above is simplified; two refinements matter a great deal in practice:

- **Static linking** copies the code of every library function you use directly into your executable at link time. The result is self-contained but larger, and does not benefit when the OS wants to share one copy of, say, `libc`, across many running programs.
- **Dynamic linking** instead records, in the executable, only the *names* of needed shared libraries (`.so` on Linux, `.dll` on Windows, `.dylib` on macOS). A specialized program, the **dynamic linker/loader** (`ld.so` on Linux), runs as the executable starts, finds each shared library already resident in memory or maps it fresh, and **patches in** the real addresses of library functions — a process called **relocation** — before your `main` function ever runs. This is why dynamically linked executables are smaller on disk but have slightly higher startup latency and a runtime dependency on the right library versions being present (colloquially, "DLL hell").
- **Position-Independent Code (PIC)** is machine code generated so it can be loaded at *any* memory address, not a fixed one — essential for shared libraries (which may be mapped at different addresses in different processes) and for security features like ASLR (Address Space Layout Randomization). Generating PIC is a back-end code-generation decision, another example of how deep the "compiler" choices reach into topics normally considered OS territory.

Extra Depth: Build Systems Are Compiler Orchestration at Scale

Everything above describes compiling *one* file. Real projects have thousands. The tools that decide *which* files need recompiling and in what order are themselves built on the phase model above:

- **Make** / **Ninja** track file modification timestamps and a dependency graph to avoid recompiling unchanged files — essentially caching the output of the "compiler" node in the pipeline diagram above, keyed by source-file identity and timestamp.
- **ccache** / **sccache** go further: they hash the *preprocessed* source (post-preprocessor, pre-compiler) plus compiler flags, and cache the resulting object file, so an unchanged file compiled with unchanged flags never re-invokes the compiler proper at all — directly exploiting the preprocessor/compiler boundary described earlier in this section.
- **Bazel and other hermetic build systems** extend caching across an entire team or CI fleet: if any engineer, anywhere, has already built a given (source, flags, toolchain) combination, everyone else can fetch the cached object/executable instead of recompiling.
- **Link-Time Optimization (LTO / ThinLTO)** deliberately *breaks* the strict separate-compilation model: instead of each `.c` file being optimized in isolation before linking, LTO defers whole-program optimization to link time, when the optimizer can see every

translation unit at once and, e.g., inline a function across what used to be file boundaries. This directly trades the modularity of the pipeline in §1.5 for better runtime performance.

Real Object-File Formats – Beyond This Introduction

This lecture's treatment of object files, linking, and loading is deliberately introductory. Real object-file formats – ELF on Linux, Mach-O on macOS, PE on Windows – each encode the relocatable code/data, symbol tables, and section layout described above in a fully specified binary format, with their own tools (`readelf`, `otool`, `dumpbin`) for inspecting them. That level of detail is not covered in this course, but is worth exploring on your own with the `objdump`/`readelf` commands mentioned above.

1.6 Different Kinds of Compilers

[beyond DB] "Compiler" is used as an umbrella term across the industry for several genuinely different tools, distinguished by *what kind of language* sits on each side of the translation (high-level vs. low-level) and *which machine* runs the result. Six worth knowing by name:

Kind	Source → Target	Example(s)
Native (AOT) compiler	High-level → machine code, same target as the host machine	GCC, Clang, `rustc` compiling for the machine they run on
Source-to-source compiler (transpiler)	High-level → a *different* high-level language	TypeScript → JavaScript; Babel (modern JS → older JS); CoffeeScript → JavaScript
Cross-compiler	High-level → machine code for a target *different* from the host running the compiler	Compiling ARM binaries for a phone on an x86 laptop; embedded firmware toolchains
Binary translator	Low-level (one instruction set) → low-level (a *different* instruction set)	Apple's Rosetta 2 (x86-64 → ARM64); QEMU's user-mode emulation
Decompiler	Low-level (machine code or bytecode) → high-level-*like* source	Ghidra, IDA Pro (machine code); JD-GUI, CFR (Java bytecode)
Bootstrapping (self-hosting) compiler	A compiler for language X, *written in* X itself	GCC (mostly C); `rustc` (written in Rust)

Example: Two of These, Worked Through

Binary translation (Rosetta 2): when Apple moved Macs from Intel (x86-64) to Apple Silicon (ARM64) in 2020, millions of existing apps were compiled x86-64 binaries with no source available to the end user. Rosetta 2 translates those already-compiled x86-64 binaries into ARM64 machine code – notice both sides of this translation are *low-level*, which is what makes

it a binary translator rather than an ordinary compiler.
Bootstrapping (GCC): you cannot compile the very first C compiler *using* a C compiler, since none exists yet. Historically, an initial, minimal C compiler was written in assembly or an even earlier language; once that minimal compiler could compile *a subset* of C, a fuller C compiler was written in C and compiled by the minimal one; that fuller compiler then compiles *itself* (and every subsequent version) from then on. This chicken-and-egg-breaking process is exactly what "self-hosting" means.

Exam Focus

Given a described tool ("translates Kotlin to Java source," "translates ARM machine code back into readable pseudo-C," "compiles Swift for an iPhone from a Mac"), be able to name which of these six kinds it is and justify your answer using the *source* → *target* column above – this classification-by-example is a very common short-answer format.

Where the Compiler-vs-Interpreter and Kind-of-Compiler Axes Meet

These two classification schemes are independent and can combine freely. A JIT compiler *is* a native compiler by this section's scheme (high-level/bytecode → machine code), just one whose *timing* (during execution) differs from an AOT native compiler's. A transpiler like Babel is almost always itself an AOT tool – it finishes its work entirely before the JavaScript it produces ever runs. Don't confuse "what kind of translation is this" (this section) with "when does the translation happen" (§1.2) – exam questions sometimes test both axes on the same tool deliberately.

1.7 A Preview: What's Next

Lecture 2 opens up the compiler "black box" from §1.2 of this lecture and walks through its internal phases – lexical analysis, parsing, semantic analysis, intermediate code generation, optimization, and final code generation – in real depth, including how the phases group into a front-end, middle-end, and back-end. Everything in today's lecture is about *what* these systems do from the outside; Lecture 2 begins answering *how*, in detail.

1.8 Self-Check Questions

Practice – Not Graded

1. In one sentence, explain why "a compiler executes your program" is a factually incorrect statement, and what does execute it instead.
2. Classic CPython has no JIT; PyPy does. Predict, with a reason, which one would be faster for a program consisting of a single loop that runs 50 million times, versus a script that runs once and exits immediately.
3. Explain, using the $M + N$ vs. $M \times N$ argument, why compiling to JVM bytecode makes Java portable in a way that compiling directly to x86 machine code would not.
4. A program compiles successfully with no errors, but fails at link time with "undefined reference to `compute_total`." Which tool caught this, and why couldn't the compiler have caught it earlier?
5. Is a transpiler (e.g. TypeScript → JavaScript) a compiler under DB's definition? Justify

your answer using the definition given in §1.1, and separately, name which of the six "kinds of compiler" from §1.6 it belongs to.

6. Give one example, not listed in this lecture, of a real tool on your own laptop that is best described as an "interpreter" rather than a compiler.
7. HotSpot's C2 JIT only compiles a method after it has been *interpreted* many times. Why not just JIT-compile every method the very first time it's called?

Key Takeaways

- A **compiler translates**; it does not itself execute the source program. An **interpreter** executes the source directly, with no independent target program left behind.
- Compilers trade slower build time for faster, repeatable run time, at the cost of a target binary that is **not portable** (tied to one ISA/OS) and paid for by the whole-program visibility that enables aggressive, multi-pass optimization. Interpreters trade that away for instant startup, easy interactivity, and **source-level portability**, at the cost of repeated re-interpretation overhead on every run.
- Most real systems today (JVM, V8, .NET, PyPy) are **hybrids**: compile to portable bytecode, interpret it at first, then **JIT-compile** the hot paths to native code at run time – Java's own history (pure interpreter in 1.0, HotSpot's tiered JIT from 1.3 onward) is the clearest real example, and it's what makes "write once, run anywhere" work.
- A JIT's big edge over AOT is **using real runtime profile data** for speculative optimization; its costs are **warm-up latency** and **runtime profiling/compilation overhead** – know this trade-off table cold.
- The compiler proper sits inside a larger **language-processing system**: preprocessor → compiler → assembler → linker → loader. The assembler produces an **object file** (relocatable code plus defined/referenced symbol tables); the **linker** resolves symbols and relocates addresses; the **loader** places the program in memory and starts it.
- "Compiler" covers several genuinely different tools: **native/AOT compilers**, **source-to-source transpilers**, **cross-compilers**, **binary translators**, **decompilers**, and **bootstrapping/self-hosting compilers** – know the source→target shape of each.
- Compiler techniques (automata, grammars, data-flow analysis) show up far outside traditional compilers – in IDEs, static analyzers, database engines, and ML frameworks – which is the real reason this course is core, not elective.
- DB §1.5's five application areas (language implementation, architecture optimization, architecture design, program translation, productivity tools) have since been joined by ML compilation, WebAssembly, and shared infrastructure like LLVM.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 2

Structure of a Compiler: Phases Overview; Front-/Middle-/Back-End; MLIR and Real-World IRs

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§1.2 (The Structure of a Compiler), §2.1–§2.8 (overview of a syntax-directed translator's stages)
Other references:	[PP] Programming Language Pragmatics §1.5, §1.6

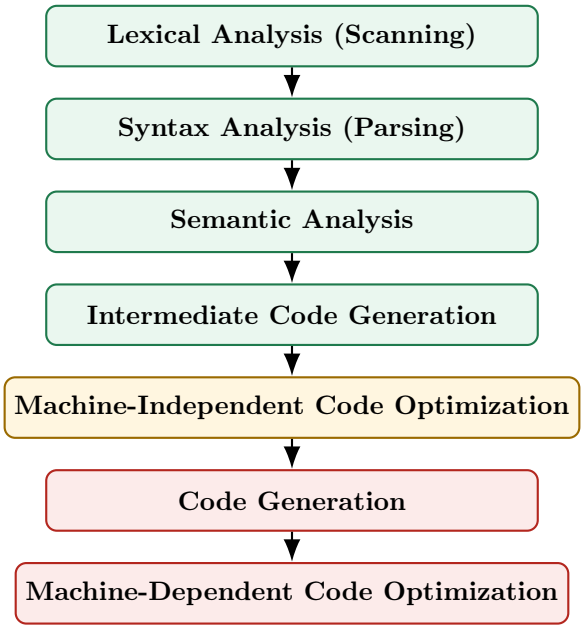
Lecture Roadmap

1. The classical phase pipeline of a compiler (DB Fig. 1.6): what each phase consumes and produces, followed all the way through on one tiny worked example.
2. The two cross-cutting components every phase touches: the **symbol table** and **error handling**.
3. Grouping the phases into **front-end** / **middle-end** / **back-end**, and why that grouping is an engineering decision, not just a diagram convenience.
4. How real-world intermediate representations – **LLVM IR, JVM bytecode, WebAssembly, and MLIR** – relate to this pipeline and to each other.

Today is still "the outside view." From Lecture 3 onward we go phase-by-phase in depth, starting with lexical analysis. (The preprocessor/assembler/linker/loader toolchain around the compiler was already covered in Lecture 1.)

2.1 The Phase Pipeline of a Compiler

[DB] DB organizes a compiler as a pipeline of phases, each of which takes the output of the previous phase as input, checks it, and produces a new representation of the program one step closer to executable code.



Phase	Input	Output
Lexical Analysis	Raw character stream	Stream of **tokens** (e.g. `ID`, `NUM`, `+`)
Syntax Analysis	Token stream	**Syntax tree** (or parse tree) capturing grammatical structure
Semantic Analysis	Syntax tree	Annotated ("decorated") tree: types checked, scopes resolved
Intermediate Code Gen.	Annotated tree	Machine-independent **IR**, e.g. three-address code
Code Optimization	IR	Improved, semantically-equivalent IR
Code Generation	Optimized IR	Target-machine code (often assembly)

Exam Focus

Memorize the phase order and, for each phase, one sentence describing its input and output. This exact table (input → phase → output) is the single most common short-answer / MCQ pattern in compiler exams worldwide, not just here.

2.1.1 A Tiny Example, Followed Through Every Phase

[DB] It helps enormously to see one small piece of source code survive the *entire* pipeline, phase by phase, before we study each phase in isolation over the coming lectures. DB uses exactly this running example when it first introduces the phases (§1.2); we reproduce a miniature version of it here, and will point back to it by name ("the `position` example") whenever a later lecture revisits one of these phases in more depth. Nothing here needs to be memorized yet – it is a preview, not new material to be tested on its own.

Source statement (assume `position`, `initial`, and `rate` are declared as real/floating-point variables, and `60` is an ordinary integer literal):

```
position = initial + rate * 60
```

Definition: Token and Lexeme

A **lexeme** is the actual sequence of characters in the source program that the lexical analyzer recognizes as a single meaningful unit – for example, the seven characters `p-o-s-i-t-i-o-n` form one lexeme, and the single character `=` forms another. A **token** is the abstract `<token-name, attribute-value>` pair the lexer emits once it has recognized a lexeme: the token-name records *which category* was matched (e.g. `ID`, `ASSIGN`, `PLUS`), and the attribute-value carries whatever extra information later phases need in order to tell this particular lexeme apart from other lexemes in the same category – for an identifier, almost always a pointer into the symbol table. Lecture 3 gives the full formal treatment, including **patterns** – the rules that describe which lexemes belong to which token.

For this statement, the **lexemes** are the individual pieces of source text `position`, `=`, `initial`, `+`, `rate`, `*`, and `60`. Lexical analysis classifies each one as a **token**: the lexeme `position` becomes `<ID, ptr to symtab entry for position>` – written `<id,1>` in the walkthrough below – and likewise `initial` and `rate` become `<id,2>` and `<id,3>`; the lexeme `=` becomes an `<ASSIGN>` token, `+` a `<PLUS>` token, `*` a `<MULT>` token, and `60` a `<NUM, 60>` token. None of the three operator tokens need an attribute-value, since the token-name alone already tells the parser everything it needs to know.

Example: The `position` Statement, Phase by Phase

- **Lexical analysis.** The character stream is grouped into tokens: `<id,1> <=> <id,2> <+> <id,3> <*> <60>` – where `id,1`, `id,2`, and `id,3` are pointers to the symbol-table entries

for `position`, `initial`, and `rate` respectively.

- **Syntax analysis.** The token stream is organized into a syntax tree that reflects the grammatical structure of an assignment: an `=` node whose left child is `id,1` and whose right child is a `+` node, which in turn has children `id,2` and a `*` node with children `id,3` and `60`.
- **Semantic analysis.** Types are checked, and where necessary, made explicit. Here, `60` is an integer literal but `rate` is real, so the semantic analyzer inserts an explicit conversion: the `*` node's right child becomes `inttoreal(60)` instead of the bare integer `60`. The result is an *annotated* ("decorated") tree.
- **Intermediate code generation.** The annotated tree is translated into three-address code, one operation per instruction:

```
t1 = inttoreal(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

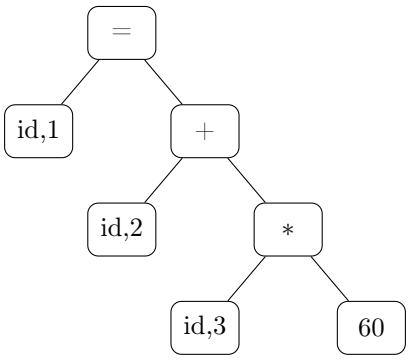
- **Machine-independent code optimization.** `inttoreal(60)` does not depend on any variable, so it can be computed once, at compile time, instead of being recomputed by the generated program every single time this line runs:

```
t1 = id3 * 60.0
id1 = id2 + t1
```

- **Code generation.** The optimized IR becomes target assembly, using real machine registers and instructions (illustrative syntax, not tied to one specific ISA):

```
MOVF   id3, R2
MULF   #60.0, R2
MOVF   id2, R1
ADDF   R2, R1
MOVF   R1, id1
```

The syntax-analysis step above builds the following syntax tree – the root `=` node assigns to `id,1` the value computed by its right subtree:



Syntax tree for `position = initial + rate * 60`.

Exam Focus

This example is deliberately small enough to hold in your head. When a later lecture introduces the *real* algorithm for a phase (Thompson's construction, LALR(1) table construction, available-expressions data-flow analysis, and so on), try re-running that algorithm on this same statement

by hand before tackling a harder one – it is a fast way to sanity-check that you have understood a new algorithm correctly.

2.1.2 Analysis and Synthesis: The Two Halves

[DB] DB groups these six phases into two conceptual halves:

- **Analysis part ("front end"):** lexical, syntax, and semantic analysis, plus intermediate code generation. This half *breaks down* the source program and builds an intermediate representation plus a record of the program's structure (the symbol table). Analysis is where most **compile-time errors** are caught and reported.
- **Synthesis part ("back end"):** optimization and code generation. This half *constructs* the desired target program from the intermediate representation and symbol table.

Definition: Analysis-Synthesis Model

A compiler can be understood as consisting of an **analysis** part, which breaks the source program into pieces and builds an intermediate representation, and a **synthesis** part, which constructs the target program from that intermediate representation. This is DB's own high-level characterization of *every* compiler, regardless of source or target language.

2.2 Symbol Table and Error Handling: The Cross-Cutting Phases

[DB] Two components are not steps in the pipeline — they run *alongside every phase*, being read from and written to throughout the whole compilation: the **symbol table** and **error handling**.

Definition: Symbol Table

A **symbol table** is a data structure that records every identifier declared in a program – variables, function names, class names – together with the attributes later phases need to know about it: its **type**, **scope**, **memory location**, and, for functions, its parameter and return types. It is not itself one of the six phases; instead every phase reads from it and writes to it: lexical analysis creates an entry the first time it sees a new identifier, semantic analysis fills in type information once it is known, and code generation consults it to compute final memory addresses.

Example: Symbol Table Contents for the `position` Example

For `position = initial + rate * 60` (§2.1.1), lexical analysis creates one entry per distinct identifier – exactly the `id,1`, `id,2`, and `id,3` pointers used throughout that walkthrough:

Entry	Name	Type	Role in this statement
`id,1`	`position`	real	assigned to
`id,2`	`initial`	real	read
`id,3`	`rate`	real	read

Notice `60` never gets its own entry – it is a numeric *literal*, not an identifier, so it is carried as a token attribute (`<NUM, 60>`) rather than being recorded in the symbol table.

Error handling. Each phase may encounter errors specific to its own level: lexical analysis catches malformed tokens, syntax analysis catches grammatically invalid programs, semantic analysis catches type mismatches and undeclared identifiers. A well-built compiler tries to *recover* from an error and keep going, so it can report several distinct errors from a single compilation run instead

of stopping at the first one.

2.2.1 Lexical, Syntax, and Semantic Errors

[DB] The three kinds of errors above are easy to name but easy to blur together in practice. Getting the distinction right matters because it tells you *which phase* is responsible for catching a given mistake – and, just as importantly, which phases are *not* responsible for it and will happily let it through.

Definition: Lexical, Syntax, and Semantic Errors

- A **lexical error** occurs when the lexer meets a character sequence that matches *no pattern for any token at all* – e.g. an illegal character, an unterminated string or comment, or a malformed numeric literal. The lexer fails before the parser ever sees this part of the input.
- A **syntax error** occurs when the token stream – with every individual token itself perfectly valid – does not conform to the grammar: e.g. a missing semicolon, an unbalanced parenthesis, or an operator with no right-hand operand.
- A **semantic error** occurs when the program is grammatically well formed but violates a rule about *meaning* that the language enforces: e.g. using an undeclared identifier, assigning incompatible types, or calling a function with the wrong number of arguments.

Each is caught by a different phase, in this order, because each phase can only see what the phase before it has already built: lexical analysis sees individual characters, syntax analysis sees the shape of the token stream, and semantic analysis is the first phase that understands what the program actually *means* (using the symbol table above to check declarations, types, and scope).

Example: One Error of Each Kind, in One Function

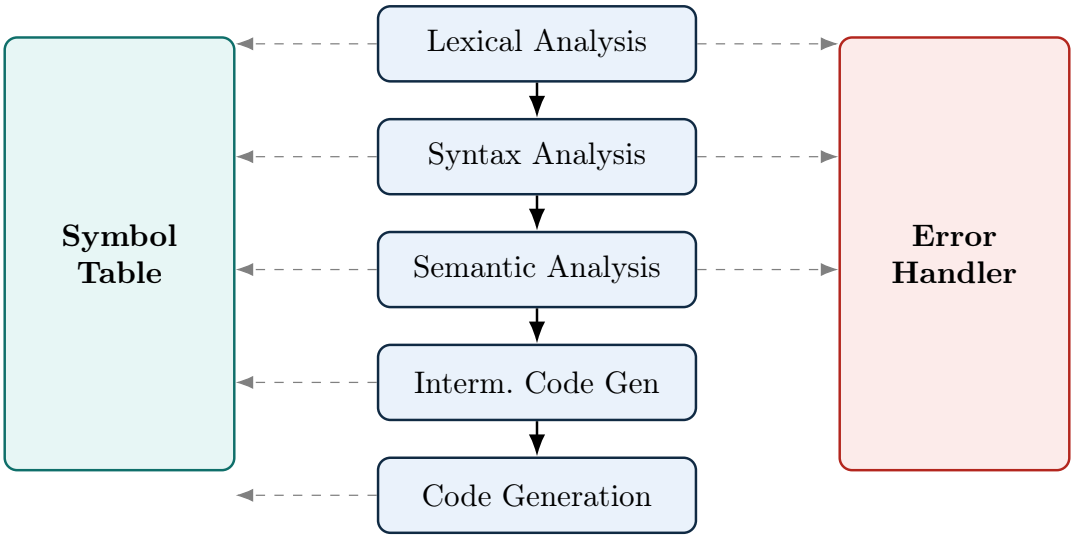
```
int main() {
    int count = 0;
    int limit = 10           /* (1) */

    while (count < limit) {
        count = count # 1;   /* (2) */
        total = total + count;   /* (3) */
    }

    printf("Final count = %d", count);
    return 0;
}
```

Line	Error Kind	Why
(1)	Syntax	`int limit = 10` is missing the terminating `;`. Every token on this line is individually a valid token – the parser is the one that fails, because it cannot complete the grammar rule for a declaration statement without a semicolon before the next statement begins.
(2)	Lexical	`#` is not a valid character inside a C expression. No token pattern matches it at all, so the *lexer itself* fails right here – the parser never even gets a chance to see this line.
(3)	Semantic	`total` is used but was never declared anywhere in this function. Every token is valid and the statement is a grammatically perfect assignment, so only semantic analysis – which checks the symbol table for declarations and scope – can catch this.

Notice the ordering this implies: if line (2)'s illegal character were fixed but line (1)'s missing semicolon were not, the compiler would still stop at the syntax error before ever reaching semantic analysis and reporting the undeclared `total` on line (3) – each phase only gets to run once the phase before it has accepted its input.

Exam Focus

Given a short, broken code snippet, be able to classify each mistake as lexical, syntax, or semantic, and justify the classification the way the table above does – this is one of the most common short-answer formats for this topic. A frequent trap: a misspelled *keyword* (like `fi` for `if`) is **not** a lexical error, because it is still a perfectly valid lexeme for token `ID` – the mistake only surfaces later, as a syntax error (if `fi` appears somewhere no identifier is grammatically valid) or a semantic error (if it is used as if it were a declared function or variable).



The symbol table is read and written by every phase, but only lexical, syntax, and semantic analysis report to the error handler – those are the three error categories this course teaches (§2.2.1). Intermediate code generation and code generation run only once the tree is already fully checked and error-free, so they are not shown wired to the error handler here.

Extra Depth: What Real Symbol Tables Store

A production compiler's symbol table stores far more than DB's introductory description suggests: mangled names (for overload resolution in C++/Rust), calling-convention information, alignment and padding requirements, visibility/linkage (`static` vs. exported), inlining hints, and — in languages with generics/templates — a separate table per instantiation. Modern compilers often implement it not as one flat table but as a **chain of nested scopes** (a scope stack or scope tree), so that entering a block, function, or class pushes a new scope that shadows outer ones and is popped on exit. LLVM's `clang` keeps this information in its `Decl` hierarchy rather than a single table, reflecting how much richer real symbol-table designs are compared to the simple table DB introduces.

2.3 Grouping the Phases: Front-End, Middle-End, Back-End

[beyond DB] DB's own diagram groups phases into analysis/synthesis, but the vocabulary you will hear constantly in industry and in later lectures is **front-end** / **middle-end** / **back-end**. This is not just relabeling — it reflects a real engineering strategy.

Group	Contains / Responsibility
Front-end	Lexical, syntax, and semantic analysis, plus intermediate code generation. Language-*specific*: knows the grammar and type rules of exactly one source language, and lowers it to a common IR.
Middle-end	Machine-independent optimization on the IR. Language-*agnostic* and target-*agnostic*: the same optimization passes (constant folding, dead-code elimination, inlining) apply no matter which source language or target CPU is involved.
Back-end	Code generation and machine-dependent optimization. Target-*specific*: knows the instruction set, register file, and calling convention of exactly one target architecture.

Why This Grouping Is the Whole Point

If front-ends only ever talk to a shared IR, and back-ends only ever talk to that same shared IR, then supporting a new *source language* only requires writing one new front-end — it automatically gets every optimization and every target the IR already supports. Symmetrically, supporting a new *target CPU* only requires writing one new back-end — it automatically works for every source language already supported. Without this split, you would need to write a separate optimizer and code generator for every (language, target) *pair* — the classic "M languages $\times$ N targets" problem, needing on the order of $M \times N$ components instead of $M + N$.

A Bit of History

This $M \times N$ argument predates LLVM by decades. In the 1960s, **T-diagrams** (popularized by F. L. Bauer and others) were a standard notation for exactly this problem: a T-shaped diagram showing a compiler written in language H, translating source language S into target language T. Compiler writers used T-diagrams to reason about *bootstrapping* (compiling a new compiler using an existing one) and about how many compilers you actually need to support M languages on N machines — precisely the front-end/back-end decoupling argument above, expressed sixty years earlier with pen-and-paper diagrams instead of shared IR software.

Extra Depth: The Front/Middle/Back Split in a Real Toolchain — LLVM

The clearest concrete instance of this architecture today is LLVM, and it is worth walking through once here since we will return to it in Lecture 25 for code generation proper.

- **Front-end (Clang, for C/C++/Objective-C):** lexes and parses source into a Clang-specific Abstract Syntax Tree (AST), performs semantic analysis and type checking on that AST, then *lowers* it into **LLVM IR** — a typed, SSA-based (Static Single Assignment) intermediate representation that is completely independent of C++ as a language. Rust (`rustc`), Swift, Julia, and Zig all have their own, completely different front-ends, but they all emit the *same* LLVM IR.
- **Middle-end (the LLVM optimizer, `opt`):** a pipeline of dozens of independent **passes** — e.g. `mem2reg` (promote stack variables to SSA registers), `instcombine` (peephole-simplify instruction sequences), `inline` (function inlining), `gvn` (global value numbering, subsuming common subexpression elimination), `loop-vectorize` — each of which reads LLVM IR and produces improved LLVM IR. None of these passes know or care whether the original source was C++, Rust, or Swift.
- **Back-end (`llc`, or the integrated code generator):** lowers optimized LLVM IR to a specific target's machine code — x86-64, AArch64, RISC-V, or even WebAssembly, selected via a `-target` triple. Instruction selection, register allocation, and instruction scheduling all happen here, specific to that one target.

The practical payoff: when Apple Silicon (AArch64) needed a good C++ compiler, no one had to write a new C++ front-end — Clang's C++ front-end and every middle-end optimization pass were already reusable; only a back-end for AArch64 was needed (and it mostly already existed, since LLVM had supported ARM for years). This is exactly the $M + N$ payoff from the coreidea box above, made concrete: roughly 20+ front-ends (Clang, `rustc`, Swift, Julia, ...) and roughly 20+ back-ends (x86-64, AArch64, RISC-V, WebAssembly, ...) share *one* middle-end, instead of each of those 20×20 combinations needing its own hand-written optimizer.

Extra Depth: The Front-End's Second Career — IDEs and Language Servers

The front-end (lexer + parser + semantic analyzer) is valuable even when no target code is ever generated. This observation underlies the **Language Server Protocol (LSP)**: an editor like VS Code or Neovim talks to a background process (a "language server") that re-runs a language's front-end on every keystroke to provide autocomplete, jump-to-definition, and real-time error squiggles — all without ever reaching code generation. `clangd` (built directly on Clang's front-end), `rust-analyzer`, and `gopls` are all, in effect, "compilers with the back end removed and an editor bolted on instead." This is a direct, practical payoff of the front-end/back-end separation described above: a front-end built for compilation can be repurposed wholesale for tooling.

2.4 Comparing Real-World IRs: LLVM IR, JVM Bytecode, and WebAssembly

[beyond DB] LLVM IR, JVM bytecode, and WebAssembly are all, loosely, "intermediate, machine-independent instruction sets sitting between source code and a real CPU" — which is why students often lump them together as interchangeable "bytecode." They are not interchangeable: each was designed with a different primary goal in mind, and that goal shapes how high- or low-level it is, what it assumes about memory and objects, and what it is actually good for.

	LLVM IR	**JVM Bytecode**	**WebAssembly (Wasm)**
Primary design goal	A reusable *compiler middle-end*: one optimizer, shared by many front-ends and back-ends	*Portability* ("write once, run anywhere") for Java-family, object-oriented, garbage-collected languages	*Safe, sandboxed* execution of code from an untrusted source (originally: the web), at near-native speed
Abstraction level	Low — close to assembly, but target-independent; typed and SSA-based	Medium-to-high — bakes in classes, interfaces, virtual method dispatch, exception tables	Low — a compact, portable, stack-based instruction set, close to assembly
Assumes an object model / GC?	No. No built-in notion of classes or garbage collection; each front-end supplies its own	Yes. Assumes the JVM supplies garbage collection and an object model	No, in Wasm's core spec. A separate, later "GC proposal" adds optional managed references
Control flow	Ordinary basic blocks and branches, the same shape as real machine code	Ordinary jumps/branches; the bytecode *verifier* checks type-safety, not control-flow shape	**Structured only** — blocks, loops, ifs; no arbitrary `goto`, so a validator can check it in a single linear pass
Usually executed directly?	No — almost always compiled further, to native code; not normally shipped as the final artifact users run	Yes — interpreted and/or JIT-compiled directly by a JVM	Yes — interpreted and/or JIT/AOT-compiled directly by a Wasm runtime
Best fit for	Being a shared layer that *very different* source languages (systems, functional, managed, ...) can all lower into	Languages that already fit the JVM's object/GC model well (Kotlin, Scala, Clojure, Groovy)	Any source language willing to compile to a small, safe, sandboxed core – often reached *via* LLVM's own WebAssembly back-end

Why LLVM IR's Low Level Is a Feature, Not a Limitation

LLVM IR deliberately does *not* assume any particular memory-management strategy or object model. That is exactly why it can serve as the shared middle layer for languages with wildly different runtime models: C's manual memory management, Rust's ownership/borrow-checked memory, Swift's automatic reference counting, and Julia's dynamic dispatch all lower into the same LLVM IR, because none of those choices had to be baked into the IR itself – each front-end encodes its own memory-management decisions *as ordinary LLVM IR instructions*. JVM bytecode makes the opposite trade-off: by assuming garbage collection and an object model up front, it becomes an excellent, low-friction target for languages that already want exactly that (Kotlin, Scala) but an awkward one for a language like C that manages memory itself. Neither design is "better" in general – they optimize for different things, which is precisely the point of this comparison.

Example: WebAssembly as "Just Another LLVM Back-End"

Because LLVM already has back-ends for x86-64, AArch64, and RISC-V, adding a WebAssembly back-end fits the exact same $M + N$ story from earlier in this lecture: any existing LLVM front-end (Clang for C/C++, `rustc` for Rust) can already produce WebAssembly simply by selecting the `wasm32` target, with *no* changes to the front-end or to any optimization pass. This is why, in practice, a great deal of WebAssembly in the wild is not hand-written but is the output of compiling C, C++, or Rust source through ordinary LLVM tooling (e.g. Emscripten for C/C++) – WebAssembly is simultaneously a stand-alone IR in its own right *and* one more entry in LLVM's list of back-end targets.

Exam Focus

Do not confuse the two axes this section and the earlier "kinds of IR" discussion test separately: *how low-level is it, and why* (this section) versus *when is it executed relative to run time* (Lecture 1's AOT/interpreter/JIT distinction). LLVM IR and WebAssembly are both low-level, but for different reasons – LLVM IR is low-level so that generating real machine code from it is easy and so that no language's runtime assumptions get baked in; WebAssembly is low-level *and* structured so that it can be validated quickly and executed safely in a sandbox. JVM bytecode is higher-level than both, by design, because portability for one specific family of managed languages – not raw compiler-infrastructure reuse, and not sandboxed safety for arbitrary source languages – was its original goal.

2.5 MLIR: A Framework for Building Compiler IRs

MLIR ("Multi-Level Intermediate Representation") is a compiler-infrastructure project started at Google around 2019 and merged into the official LLVM monorepo as one of its sub-projects in 2020, first described publicly in [MLIR]. It is worth understanding alongside LLVM IR for the same reason the previous section compared LLVM IR, JVM bytecode, and WebAssembly: people often assume "MLIR" is just another name for "LLVM IR, but newer," when in fact the two solve different problems and sit at different points in a compiler's pipeline.

2.5.1 The Problem MLIR Was Built to Solve

LLVM IR is deliberately low-level: typed, in SSA form, organized into ordinary basic blocks – close to assembly, as §2.4 described. That is exactly what makes it a good shared target for many different *source* languages once they have already been reduced to simple operations and control flow. It is a poor fit, however, for representing a program *before* that reduction has happened – for example, a matrix multiplication expressed as a single high-level tensor operation, or a nested loop that a compiler still intends to tile, fuse, or vectorize as a unit. Lowering straight to LLVM IR forces that structure to be flattened away immediately, before any optimization pass gets a chance to use it.

Before MLIR existed, projects that needed this kind of higher-level representation simply built their own IR from scratch: TensorFlow's XLA had its own IR for tensor computations, Swift had SIL, Rust had MIR, and so on. Each of these reimplemented the same underlying machinery – a verifier, a textual printer and parser, a pass manager, a way to track source locations for error messages – that LLVM already provides for LLVM IR, but for their own, unrelated IR. MLIR's goal is to give every such project a shared framework for this machinery, so that building a new, domain-specific IR is a matter of defining new operations rather than rebuilding compiler infrastructure from the ground up.

2.5.2 Core Ideas

Dialects. Instead of one fixed instruction set, MLIR defines an extensible mechanism for registering **dialects** – namespaced families of operations, types, and attributes. The `arith` dialect has operations like `arith.addi`; the `linalg` dialect has structured, high-level operations such as matrix multiplication; the `scf` dialect ("structured control flow") has `for` and `while` loops as single operations, rather than as jumps between basic blocks; and the `llvm` dialect defines operations that directly mirror LLVM IR. Crucially, operations from several different dialects can appear together in the very same function – there is no requirement to fully translate out of one dialect before using another.

Progressive lowering. A program does not jump from a high-level dialect straight to machine code in one step. It is lowered dialect by dialect: a high-level tensor operation becomes a structured loop, the structured loop becomes ordinary branching control flow, and that, in turn, becomes the `llvm` dialect. Each of these steps is a small, independently checkable transformation, and different compilers can choose to stop lowering at whichever level is most useful for a given optimization – a loop-tiling pass wants to run before loops are flattened into branches, for instance, while register allocation only makes sense long after that point.

Regions. An ordinary LLVM IR function is a flat sequence of basic blocks connected by branches – there is no notion of one block being "inside" another. An MLIR operation, by contrast, can carry a nested **region** containing its own blocks. A loop or a function body is therefore naturally represented as one operation containing a region, preserving its nested structure for as long as that structure remains useful, instead of being flattened into a control-flow graph immediately.

2.5.3 How MLIR Differs From LLVM IR

	LLVM IR	**MLIR**
What it is	One fixed intermediate representation	A framework for defining many intermediate representations (dialects)
Abstraction level	Low and fixed – close to assembly	Whatever a given dialect needs – high, low, or several levels mixed in one module
Control-flow structure	Flat basic blocks connected by branches	Operations can carry nested regions, so loops and other structure can stay nested until a pass chooses to flatten them
Role in a pipeline	The representation code is lowered *to*, immediately before LLVM's own optimizer and back-ends take over	Sits *upstream* of LLVM IR; its own `llvm` dialect is what eventually converts into ordinary LLVM IR

The last row is the point most often missed: MLIR does not replace LLVM. A program written using MLIR's higher-level dialects is progressively lowered, within MLIR, down to the `llvm` dialect; from there it is translated into ordinary LLVM IR and handed to the exact same optimizer passes and back-ends already described in §2.3 and §2.4. MLIR extends the *front* of the pipeline – giving compiler writers a place to do domain-specific reasoning (tensor shapes, loop tiling, hardware-specific fusion) before code is flattened down to LLVM IR – rather than replacing anything downstream of it.

2.5.4 Where MLIR Is Used

MLIR's dialect mechanism has made it attractive well beyond its original use case. Notable adopters include Google's XLA and IREE (compiling machine-learning computation graphs to CPUs, GPUs,

and custom accelerators), PyTorch's Torch-MLIR, Triton (a language for writing GPU kernels), CIRCT (an MLIR-based toolchain for hardware design, playing a role similar to parts of a Verilog toolchain), and Flang, LLVM's own Fortran front end, which lowers through an MLIR dialect called FIR before reaching LLVM IR (see [MLIR, MLIR-DOCS] for further reading).

2.6 Self-Check Questions

Practice – Not Graded

1. Place the following in correct pipeline order: semantic analysis, linking, assembly, lexical analysis, loading, syntax analysis, preprocessing, code generation.
2. A program calls a function defined in a library it never `#includes` a declaration for, but the linker still resolves the call successfully because the symbol name matches at link time. Which phase would have caught this as an error if the declaration truly were missing, and why didn't it?
3. Explain, in the M-languages/N-targets argument, why a shared IR turns an $M \times N$ problem into an $M + N$ problem. What is the "shared IR" in LLVM's case?
4. Why does dynamic linking trade smaller executables for slower startup, while static linking does the opposite?
5. `clangd` never performs code generation. Which phases of the pipeline does it still need to run, and why are those enough to power features like "jump to definition"?
6. For the `position` example in §2.1.1, write out what the three-address code would look like if `60` were replaced by `count`, an already-integer variable (so no `inttoreal` conversion is needed). Which of the six phases changes as a result, and which stay exactly the same?
7. JVM bytecode assumes garbage collection; LLVM IR does not. Explain why this makes JVM bytecode a natural fit for Kotlin but a poor natural fit for a language like C, and explain why WebAssembly's core spec makes the same choice as LLVM IR here rather than the JVM's.
8. For the statement `count = count + step`, list its lexemes, give the `<token-name, attribute-value>` pair for each one, and sketch the symbol table entries lexical analysis would create (assuming `count` and `step` are both integers).
9. Draw the syntax tree for `count = count + step` in the same style as the `position` example's tree, and identify which single node the semantic analyzer would need to annotate if `step` were declared `real` instead of integer.
10. For each of the following, classify the mistake as a lexical, syntax, or semantic error, and justify your answer in one sentence: (a) `int x = 5$;` (b) `if (x > 0 { y = 1; }` (missing a closing parenthesis) (c) calling `compute(x, y)` when `compute` was declared to take only one parameter.

Key Takeaways

- A compiler's six classical phases, in order: lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, code generation.
- These split into an **analysis** half (front end: builds an IR and symbol table) and a **synthesis** half (back end: builds the target program from that IR).
- The `position = initial + rate * 60` example shows all six phases acting on one tiny statement: tokens → syntax tree → annotated tree (type conversion inserted) → three-address code → optimized three-address code (constant folded) → target assembly.

- A **token** is an abstract `<name, attribute>` pair emitted for a recognized **lexeme** (the actual matched source text). A **symbol table** records one entry per declared identifier and is read from and written to by every phase – not a phase itself.
- **Lexical, syntax, and semantic errors** are caught by three different phases, in that order: a lexical error means no token pattern matches at all (e.g. an illegal character); a syntax error means every token is valid but their arrangement breaks the grammar (e.g. a missing semicolon); a semantic error means the program is grammatically fine but violates a meaning-level rule the language enforces (e.g. an undeclared identifier). A misspelled keyword is usually *not* a lexical error, since it is still a valid identifier lexeme.
- The **front-end/middle-end/back-end** split (language-specific $\rightarrow$ language-and-target-agnostic $\rightarrow$ target-specific) is what lets one shared IR (like LLVM IR) turn an $M \times N$ language/target problem into an $M + N$ problem.
- **LLVM IR, JVM bytecode, and WebAssembly** are all machine-independent instruction sets but were built for three different goals – reusable compiler infrastructure, portability for GC'd object-oriented languages, and safe sandboxed execution, respectively – and that goal explains how high- or low-level each one is and what it assumes about memory and objects.
- **MLIR** is not a fixed IR but a framework for defining many **dialects** at different abstraction levels, progressively lowered down to LLVM IR – it extends the front of the pipeline rather than replacing anything downstream of it.
- Front-ends have value beyond compilation: language servers (`clangd`, `rust-analyzer`) repurpose the same lexer/parser/semantic-analyzer pipeline for IDE tooling, without ever reaching code generation.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.

[MLIR] Lattner, C., Amini, M., Bondhugula, U., Cohen, A., Davis, A., Pienaar, J., Riddle, R., Shpeisman, T., Vasilache, N., and Zinenko, O. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. arXiv:2002.11054, 2020.

[MLIR-DOCS] The MLIR Project. *Official Documentation*. https://mlir.llvm.org/

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 3

Lexical Analysis: Tokens, Patterns, and Lexemes; the Chomsky Hierarchy; Regular Languages

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.1 (The Role of the Lexical Analyzer), §3.3 (Specification of Tokens: terminology, strings, languages, operations)
Other references:	[AP] Modern Compiler Implementation, Ch. 2

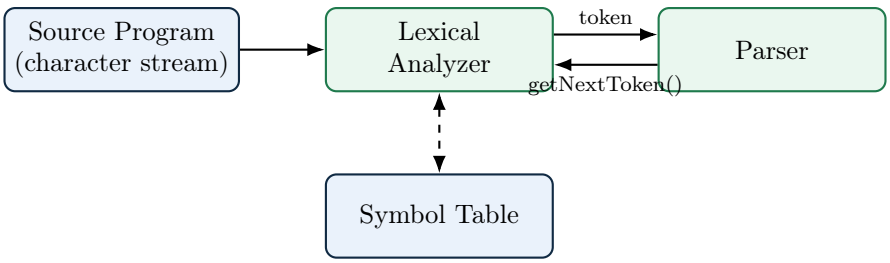
Lecture Roadmap

1. Where lexical analysis sits, and exactly how it talks to the parser; why lexer and parser are separate phases.
2. The three core vocabulary words of this unit: **token**, **pattern**, **lexeme** – with several fully worked examples, plus token **attributes** and **lexical errors**, including a close look at **panic-mode recovery**.
3. **The big picture**: how a regular expression eventually becomes a working, table-driven lexer, via finite automata – the roadmap for this lecture and the ones after it.
4. The **Chomsky hierarchy**: how formal, regular, context-free, and more powerful language classes relate to each other, and which machines recognize each one.
5. **Regular languages**, defined formally and precisely – the last topic of today, and the foundation the next lecture builds **regular expressions** on top of.

By the end of this lecture you will know exactly which languages qualify as "regular" and why, and you will have the full roadmap for how such a language eventually becomes a running lexer. Lecture 4 picks up immediately from here: regular expressions, the notation we use to *write down* a regular language.

3.1 Where Lexical Analysis Fits

[DB] The **lexical analyzer** (also called a **scanner** or **lexer**) is the first phase of a compiler. It reads the raw stream of characters making up the source program and groups them into a stream of **tokens** — meaningful chunks like keywords, identifiers, numbers, and operators — that the rest of the compiler works with instead of raw text.



How the Lexer and Parser Actually Talk to Each Other

In nearly every real compiler, the lexer does *not* tokenize the whole file up front and hand the parser a list. Instead, the parser drives the process: whenever the parser needs another token, it calls a function conventionally named `getNextToken()`. The lexer resumes scanning exactly where it left off, produces the next single token, and returns it. This "pull" model, sometimes called treating the lexer as a **coroutine** of the parser, avoids ever materializing the full token list in memory and lets the lexer and parser interleave their work one token at a time.

3.1.1 Why Not Just One Phase?

[DB] DB gives three concrete engineering reasons for splitting lexical analysis out as its own phase rather than folding it into the parser:

1. **Simplicity of design.** Separating out low-level tasks (recognizing keywords, numbers, string literals) lets the parser work purely with grammatical structure, using tokens as its atomic symbols, instead of also worrying about individual characters.
2. **Improved efficiency.** Specialized, table-driven techniques for recognizing tokens – built from finite automata, which the lectures right after this one construct *directly* from the regular expressions we develop later in this very lecture – are considerably faster than the general-purpose parsing techniques would be if forced to operate character-by-character.
3. **Enhanced portability.** Input-device and operating-system-specific quirks — end-of-line conventions (`\n` vs. `\r\n`), character encodings, special end-of-file characters — are confined entirely to the lexer. The parser, and everything after it, is completely insulated from these differences.

Exam Focus

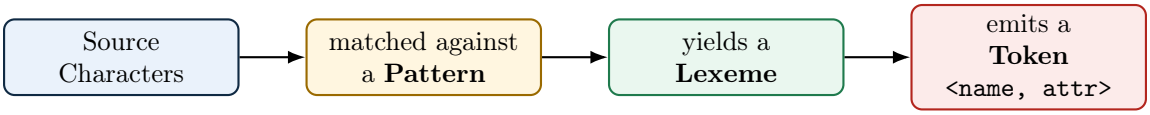
This "three reasons" list (simplicity, efficiency, portability) is a favorite short-answer question. Be able to name and briefly justify all three, not just one.

3.2 Tokens, Patterns, and Lexemes

[DB] These three terms are used precisely and are easy to blur together. Getting the distinction exactly right is one of the most exam-tested ideas in this entire unit.

Definition: Token, Pattern, Lexeme

- A **token** is a pair `<token-name, attribute-value>` representing a *category* of lexical unit, e.g. `ID`, `NUM`, `RELOP`. The token-name is an abstract symbol used by the parser.
- A **pattern** is the rule that describes the *set of possible lexemes* that can produce a given token – for identifiers, informally, "a letter followed by any number of letters and digits." Lecture 4 makes patterns precise using regular expressions.
- A **lexeme** is the actual sequence of characters in the source program that *matches* a pattern and is classified as an instance of some token.



Example: DB-Style Token/Pattern/Lexeme Table

Token	Informal Description of Pattern	Sample Lexemes
`WHILE`	the exact keyword `while`	`while`
`ID`	a letter, followed by any number of letters or digits	`i`, `sum`, `score2`
`NUM`	any numeric constant	`0`, `42`, `3.14`
`RELOP`	<, <=, =, <>, >, or >=	<=, <>
`LITERAL`	any characters between a pair of double quotes, except a double quote itself	`"Total = "`

3.2.1 Worked Example 1: Tokenizing a Function Call

Consider the following line of C-like source code:

```
printf ( "Total = %d\n" , score ) ;
```

identifier punctuation/operator string literal

Lexeme	Token Name	Attribute-Value (passed to parser)
`printf`	`ID`	pointer to symbol-table entry for `printf`
`(`	`LPAREN`	none needed
`"Total = %d\n"`	`LITERAL`	pointer to the string's stored value
`,`	`COMMA`	none needed
`score`	`ID`	pointer to symbol-table entry for `score`
`)`	`RPAREN`	none needed
`;`	`SEMI`	none needed

3.3 Attributes for Tokens

[DB] When more than one lexeme can match a token's pattern (as with `ID` or `NUM`), the lexer must pass along enough extra information – the **attribute-value** – for later phases to know exactly *which* identifier or *which* number was matched. For identifiers, this is essentially always a pointer into the symbol table.

Example: Worked Example 2: An Assignment Statement

Source line: `E = M * C ** 2`
Assume `**` is this language's exponentiation operator. The lexer's output, as a stream of `<token-name, attribute-value>` pairs, is:

```
<ID, ptr to symtab entry for E>
<ASSIGN_OP, - >
<ID, ptr to symtab entry for M>
<MULT_OP, - >
<ID, ptr to symtab entry for C>
<EXP_OP, - >
<NUM, integer value 2>
```

Notice: `ID` tokens all carry a symbol-table pointer as their attribute so that later phases can distinguish `E` from `M` from `C`; the operator tokens need no attribute at all, since the token-name

alone (`ASSIGN_OP` vs. `MULT_OP`) already says everything the parser needs to know.

Exam Focus

Given a short program line, be able to produce the exact stream of `<token-name, attribute-value>` pairs, in order, the way both worked examples above do. This is the single most common long-answer question style for this unit.

3.3.1 Worked Example 3: A Complete Statement Block

```
int sum = 0;
while (i <= 10) {
sum = sum + i;
i = i + 1;
}
```

Lexeme	Token	Attribute	Lexeme	Token	Attribute
int	TYPE	–	sum	ID	ptr(sum)
sum	ID	ptr(sum)	=	ASSIGN	–
=	ASSIGN	–	sum	ID	ptr(sum)
0	NUM	0	+	PLUS	–
;	SEMI	–	i	ID	ptr(i)
while	WHILE	–	;	SEMI	–
(	LPAREN	–	i	ID	ptr(i)
i	ID	ptr(i)	=	ASSIGN	–
<=	RELOP	LE	i	ID	ptr(i)
10	NUM	10	+	PLUS	–
)	RPAREN	–	1	NUM	1
{	LBRACE	–	;	SEMI	–
			}	RBRACE	–

Two details worth noticing in this longer example:

- `sum` and `i` each get *their own* symbol-table pointer the first time they're seen, and the *same* pointer every time they reappear – the symbol table guarantees one entry per distinct identifier, no matter how many times it's used.
- Whitespace (the spaces and newlines between tokens) and indentation produce *no* tokens at all – they are consumed and discarded by the lexer, along with any comments, before the parser ever sees anything.

The Lexer's "Housekeeping" Duties

Beyond producing tokens, DB notes the lexer is also the natural place to: strip out whitespace and comments (neither produces a token); and track line numbers (and often column numbers) so that later phases can report errors with accurate source locations, e.g. "`undeclared variable 'x' at line 42`."

3.4 Lexical Errors

[DB] It is surprisingly rare for a lexer to be able to detect an error entirely on its own, *by itself, in isolation* – because almost any short character sequence is a *prefix* of some valid lexeme for some token.

Example: The Classic Illustration

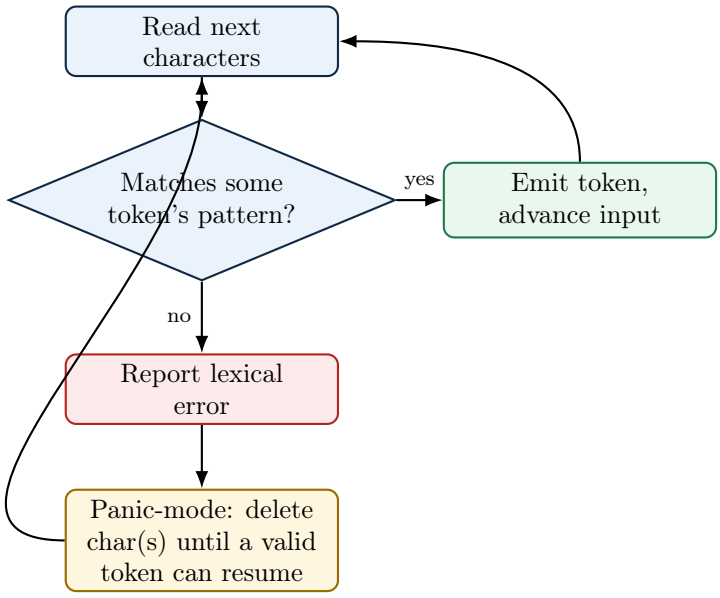
Consider the fragment `fi (a == f(x))` in a C-like language, where the programmer *meant* to type the keyword `if`. The lexer cannot know this. `fi` is a perfectly legal lexeme for token `ID` ("a letter followed by letters/digits"), so the lexer happily emits `<ID, ptr(fi)>`. The mistake will only be caught later – by the *parser*, if `fi` appears somewhere no identifier is grammatically valid, or by *semantic analysis*, if `fi` is used as if it were a function without ever having been declared.

Genuine lexical errors are ones where *no pattern for any token* can match the remaining input at all, or where a token's own internal structure is malformed:

- An illegal character that starts no valid token in the language, e.g. a stray `#` or `@` in a language that assigns those characters no meaning.
- An unterminated string literal: opening quote seen, but the line (or file) ends before a matching closing quote.
- A malformed numeric literal, e.g. `3.14.15` (two decimal points), or `2r` immediately followed by a letter with no valid meaning in that position.
- An unterminated comment: a `/*` seen with no matching `*/` anywhere before end of file.

3.4.1 Error Recovery Strategies

[DB] A production-quality lexer does not simply stop at the first error — it tries to recover and keep going, so the programmer can see *several* distinct errors from one compilation attempt rather than fixing them one at a time.



[DB] Panic mode, in detail. "Panic mode" names a very specific, very simple procedure, and it helps to spell out its steps exactly, since the name alone makes it sound more dramatic than it is: (1) report the error at the current position; (2) delete (discard) the current character; (3) check whether the *remaining* input now begins with something matching a valid token's pattern; (4) if not, go back to step 2 and delete the next character too; (5) as soon as the remaining input does match a valid token – or a natural boundary like end of line or end of file is reached – stop deleting and resume ordinary scanning from there. In short: *throw away characters, one at a time, until scanning can safely restart.*

Example: Worked Example 1: A Single Illegal Character

Source line: `total = x @ y;`
The lexer scans `total` (`ID`), `=` (`ASSIGN`), `x` (`ID`) normally. It then reaches `@`, which matches no token pattern in this language. **Step 1:** report "illegal character `@` at column 12." **Steps 2–3:** delete just the `@`; the very next character, a space, already leads into a lexeme (`y`) that matches `ID`. So panic mode stops after deleting a single character. Scanning resumes normally: `y` (`ID`), `;` (`SEMI`). One error reported, and the rest of the line recovers cleanly – this is the best case for panic mode.

Example: Worked Example 2: Discarding Until a Natural Boundary

Source:

```
printf("Hello world);
x = 5;
```

The lexer scans `printf` (`ID`), `(` (`LPAREN`), then starts matching a `LITERAL` token at the opening `"`. It looks for a matching closing `"` but reaches the end of the line without finding one. Report: "unterminated string literal starting at line 1." Unlike Worked Example 1, there is no small number of characters to delete that fixes this – *nothing* inside a broken string is a safe resumption point. Panic mode instead discards everything from the opening quote through the end of the line (a natural boundary most languages use for exactly this reason: string literals cannot normally span a raw newline), and resumes scanning fresh at line 2: `x` (`ID`), `=` (`ASSIGN`), `5` (`NUM`), `;` (`SEMI`). One error reported; the rest of the program is still checked for further problems.

Example: Worked Example 3: Why Panic Mode Can Cascade Into Extra Errors

Source line: `y = 3.14.159 + 2;`
The lexer scans `y` (`ID`), `=` (`ASSIGN`), then starts matching a `NUM` at `3.14`. Maximal munch tries to extend the match, but a *second* `.` cannot extend a valid `num` (DB's regular definition, covered fully with Lecture 4's `num` example, allows only one optional fraction part) – so the lexer accepts what it *did* successfully match, emits `<NUM, 3.14>`, and resumes scanning from the very next character: a bare `..` But a lone `.` does not start any valid token in this language either, so panic mode fires a *second* time, reporting a second illegal-character error and deleting just that `.`, before finally matching `159` as another `NUM` token. **One root mistake – a stray decimal point – surfaces as two reported errors.** This is a known, unavoidable downside of panic mode: it recovers *syntactically* (scanning can always continue), not *semantically* (it has no idea what the programmer actually meant), so one typo can occasionally produce a short burst of related-looking error messages. Experienced programmers learn to fix the *first* error in a cluster and recompile, rather than trying to fix every reported line individually.

[DB] Beyond panic mode: local corrections. DB also lists four more surgical "local-correction" strategies a recovering lexer *could* attempt instead of blindly discarding characters – though, as the note after the examples explains, only two of them apply cleanly to genuine lexical errors:

- **Deleting** an extraneous character – e.g. the malformed number `3..14` (one `.` too many): deleting the second `.` recovers the valid number `3.14`, instead of panic mode's blunter "discard until something matches."
- **Inserting** a missing character – e.g. the unterminated string `"Hello` with the line ending before a closing quote: inserting a `"` right where the line ends recovers a (short) valid string literal, letting scanning continue past it immediately instead of discarding the rest of the line as Worked Example 2 did.
- **Replacing** one character with another – e.g. `3@4` plausibly intended as `3*4`: replacing `@` with `*` recovers a valid expression.

- **Transposing** two adjacent characters – e.g. `"Helol"` plausibly intended as `"Hello"`.

Exam Focus

Not all four local corrections are equally "lexical." **Deletion** and **insertion** directly repair the two genuine categories of lexical error from §3.4 – an extra character the pattern can't accommodate, or a character the pattern is missing – so a lexer's own panic-mode-plus-local-correction logic can apply them without knowing anything about programmer intent. **Replacement** and **transposition**, by contrast, mostly turn one *valid* lexeme into a *different* valid lexeme (as the `Helol`→`Hello` example shows) – exactly the "looks like a typo but is a perfectly legal token" situation from the `fi`/`if` example earlier in this lecture, which is *not* a lexical error at all and is therefore outside what the lexer itself can or should silently fix. Replacement and transposition matter far more for IDE-style "did you mean . . . ?" suggestions layered on top of the compiler than for the lexer's own error-recovery logic.

A Bit of History

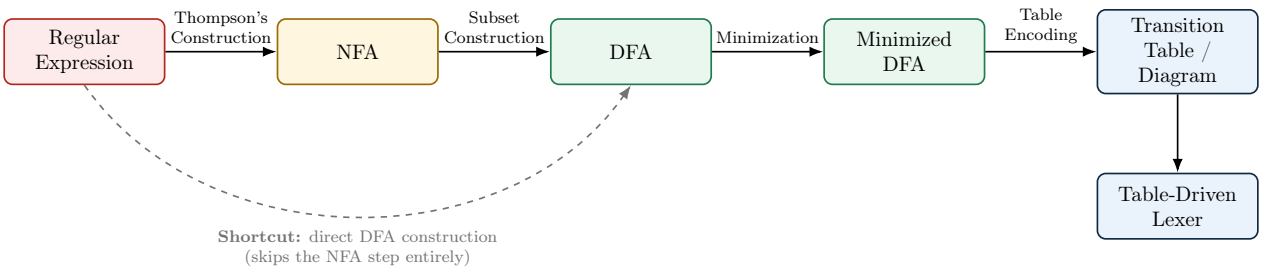
DB points toward a more principled version of this idea: choosing whichever single correction requires the *fewest* character edits – insertions, deletions, substitutions – to turn the bad input into something valid. This is exactly **edit distance** (Levenshtein distance), the same string-similarity measure used today in spell-checkers and DNA sequence alignment. Aho and Peterson described a minimum-distance error-correcting parser as early as 1972; the idea that "the best fix is the cheapest fix" turns out to generalize far beyond lexical analysis, into syntax error recovery and even runtime auto-correct systems.

Exam Focus

Be able to explain, with a concrete example like `fi (a == f(x))`, *why* a misspelled keyword is usually **not** a lexical error even though it looks like an obvious typo. Then be able to walk through panic mode step by step (report, delete, check, repeat, resume) on a short broken input the way the three worked examples above do – this is the most common long-answer version of this topic, and students often lose marks by describing panic mode too vaguely ("it skips bad characters") instead of naming the actual step-by-step procedure.

3.5 The Big Picture: From Regular Expressions to a Working Lexer

[DB] Every pattern we have used so far has been informal English: "a letter followed by letters or digits." That is fine for talking about lexical analysis, but it is not something a computer can execute. Turning informal patterns into an actual running lexer takes a short pipeline of well-defined, *mechanical* steps, and it is worth seeing the whole pipeline once, up front, before building any one piece of it.



Step	What Happens
Regular expression → NFA	**Thompson's construction**: a purely mechanical algorithm that builds a nondeterministic finite automaton (NFA) directly from the structure of a regular expression, one small automaton per operator.
NFA → DFA	**Subset construction**: another mechanical algorithm that converts any NFA into an equivalent deterministic finite automaton (DFA) – one with no ambiguity about which transition to take, at the cost of (possibly) more states.
(shortcut) Regular expression → DFA directly	Some tools skip the NFA step entirely: DB §3.9 builds a DFA *directly* from a regular expression's syntax tree, using functions traditionally called *nullable*, *firstpos*, *lastpos*, and *followpos*. It computes exactly the same DFA that Thompson's construction plus subset construction would have produced (possibly needing a separate minimization pass afterward) – it is a different *route* to the same destination, not a different destination, and it is what many real lexer generators use internally because it tends to build fewer intermediate states.
DFA → minimized DFA	**DFA minimization**: merges states that are behaviorally indistinguishable, producing the smallest possible DFA that still recognizes exactly the same language.
Minimized DFA → transition table	The (finite!) set of states and transitions is encoded as a 2-D table: `table[state][input_symbol] = next_state`.
Transition table → lexer	A tiny, fast loop – look up the current state and next character in the table, move to the next state, repeat – *is* the lexer's inner loop. No more cleverness is needed once the table exists.

Exam Focus

"Transition diagram" and "transition table" are not two different stages of this pipeline – they are two different *representations of the same DFA (or NFA)*. A **transition diagram** is the picture: circles for states, labeled arrows for transitions, exactly the automaton drawings you will see in the next few lectures. A **transition table** is the same information laid out as a 2-D array instead of a picture. Neither one is more "correct" than the other, and neither is a separate construction step – once you have a DFA, you already have both, you have just chosen how to write it down. A common exam trap asks you to convert between the two for a small automaton; that is a notation exercise, not a new algorithm.

Why This Pipeline Matters

Every solid arrow in the diagram above is a *mechanical, automatable* algorithm – none of them require human judgment once the previous step is done, and the dashed shortcut is simply an alternative route to the same DFA. This is exactly why lexer-generator tools like Flex exist: you write only the left-most box (a set of regular expressions, one per token), and the tool runs the entire rest of the pipeline for you, emitting ready-to-compile scanner code. The remaining lectures on lexical analysis build this pipeline one arrow at a time – Thompson's construction and subset construction first, then minimization – so that by the time we reach Flex itself, you understand exactly what the tool is doing on your behalf rather than treating it as a black box.

3.6 Strings, Alphabets, and Languages

[DB] Before regular expressions can be defined precisely, three primitive terms need to be pinned down exactly as DB uses them.

Definition: Alphabet, String, Language

- An **alphabet** Σ is any finite set of symbols, e.g. $\{0, 1\}$ or the set of ASCII characters.
- A **string** (or *word*) over Σ is a finite sequence of symbols drawn from Σ. The **empty string**, written ε, has length 0.
- A **language** over Σ is simply *any* set of strings over Σ — possibly infinite, possibly empty ($\emptyset$, the language with no strings at all, which is *not* the same as $\{\varepsilon\}$, the language containing only the empty string).

Exam Focus

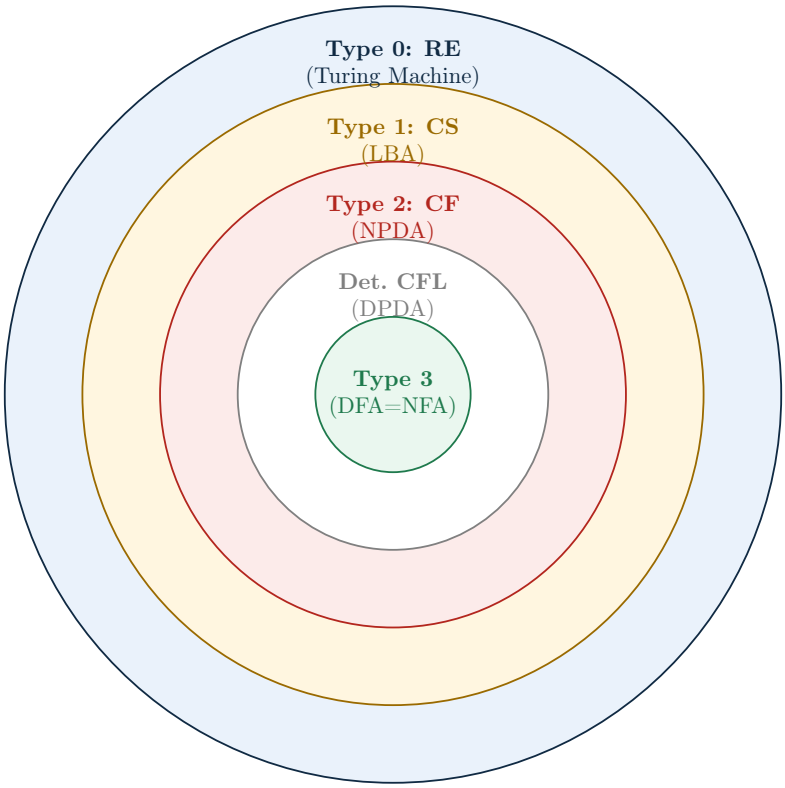
$\emptyset$ and $\{\varepsilon\}$ are a classic confusion. $\emptyset$ contains *zero* strings; $\{\varepsilon\}$ contains *one* string (the empty one). Keep them distinct.

Formal Languages: A Hierarchy, Not One Thing

The definition above – "a language is any set of strings" – deliberately puts *no* restriction on what that set looks like. A set meeting only this bare definition is often called a **formal language**, precisely to flag that nothing about its internal structure is assumed yet. Computer science studies several increasingly restrictive (and increasingly cheap to process) *classes* of formal languages, most famously organized into the **Chomsky hierarchy** – the subject of the subsection below.

3.6.1 The Chomsky Hierarchy

[DB] Linguist Noam Chomsky proposed this hierarchy in 1956 while studying the structure of natural language; computer science adopted it almost immediately, because it turns out to classify exactly the right notion for programming languages too – *how much machinery do you need to recognize this language?* Each class below is defined by a restriction on the *grammar* allowed to generate it, and each corresponds to exactly one class of recognizing machine.



RE = Recursively Enumerable, CS = Context-Sensitive, CF = Context-Free – full names and machines in the table below.

Each ring properly contains *the ones inside it: every regular language is context-free, every context-free language is context-sensitive, and so on – but not the reverse.*

Type	Class	Machine	Deterministic vs. Nondeterministic
3	Regular	DFA or NFA	**Equal power.** Every NFA has an equivalent DFA (via the *subset construction* we build toward in the next few lectures) – nondeterminism buys convenience of expression here, not extra recognizing power.
–	Deterministic CFL	DPDA	**Strictly weaker than NPDA.** Deterministic pushdown automata recognize only a proper subset of the context-free languages – this middle ring is not one of Chomsky's original four types, but is a genuinely useful refinement between Types 2 and 3.
2	Context-Free	PDA (nondeterministic, NPDA)	**Strictly stronger than DPDA.** The classic separating example: $\{ww^R : w \in \{a, b\}^*\}$ (even-length palindromes) is context-free, but no DPDA can recognize it – a deterministic machine can't "guess" where the middle of the string is without trying multiple possibilities at once.
1	Context-Sensitive	Linear-Bounded Automaton (LBA)	**Unknown!** Whether deterministic and nondeterministic LBAs have equal power is the **LBA problem**, open since 1964 – one of the oldest unsolved questions in theoretical computer science.
0	Recursively Enumerable	Turing Machine	**Equal power, and this is the ceiling.** Every reasonable model of computation ever proposed – multi-tape TMs, nondeterministic TMs, RAM machines, λ-calculus – has been proven exactly as powerful as the single-tape deterministic TM. This equivalence, and the belief that no more powerful model of *effective computation* exists, is the **Church–Turing thesis**.

Exam Focus

Keep the **NFA/DFA** case and the **PDA/DPDA** case straight – they go opposite ways, and exams like to test exactly this contrast. For *finite* automata, adding nondeterminism costs nothing in power (only in state count). For *pushdown* automata, adding nondeterminism genuinely lets you recognize more languages. This single fact is the formal reason a lexer can always be built deterministically (fast, table-driven), while a general context-free parser cannot always be – some grammars force real backtracking or lookahead machinery that a DPDA-equivalent design cannot avoid.

This lecture defines exactly one of these classes with full precision – the **regular languages**, starting below. **Context-free languages** are the parser's formalism, and the course returns to them once we reach syntax analysis; every regular language is also context-free (but not the reverse), which is exactly why a parser can express everything a lexer's regular expressions can, and more.

3.7 Regular Languages: A Formal Definition

[DB] To say precisely which languages qualify as "regular," we first need a small set of operations for building new languages out of old ones. Given two languages L and M over the same alphabet, DB defines four such operations:

Operation	Notation	Definition
Union	$L \cup M$	$\{ s \mid s \in L \text{ or } s \in M \}$
Concatenation	LM	$\{ st \mid s \in L \text{ and } t \in M \}$ (paste a string from L directly onto a string from M)
Kleene closure	L^*	$\bigcup_{i=0}^{\infty} L^i$ – zero or more strings from L, concatenated (includes ε, from L^0)
Positive closure	L^+	$\bigcup_{i=1}^{\infty} L^i$ – one or more strings from L, concatenated (excludes ε unless $\varepsilon \in L$)

Example: Worked Example: Letters and Digits

Let $L = \{a, b, c, \ldots, z, A, \ldots, Z\}$ (the 52 letters) and $D = \{0, 1, \ldots, 9\}$ (the 10 digits), each treated as a language of one-character strings.

- $L \cup D$ – every single letter *or* single digit: 62 strings, each of length 1.
- LD – every string formed by one letter followed by one digit, e.g. `a5`, `Z0` – exactly $52 \times 10 = 520$ strings, each of length 2.
- L^4 – every string of exactly 4 letters, e.g. `Test`, `abcd` – there are 52^4 such strings in total.
- L^* – every string of letters of *any* length, including ε — infinitely many strings.
- $L(L \cup D)^*$ – one letter, followed by zero or more letters-or-digits. This is *exactly* the language of identifiers in most C-like languages! We will turn this directly into a regular expression in Lecture 4.

[DB] With these operations in hand, we can now say precisely what "regular" means – a statement about the language itself, as a set of strings, before we ever write a single regular expression on paper.

Definition: Regular Language (Inductive Definition)

The **regular languages** over an alphabet Σ are defined inductively, using exactly the union, concatenation, and Kleene-closure operations above.

Base cases. Each of the following is a regular language:

(a) $\emptyset$ – the empty language, containing no strings at all;
(b) $\{\varepsilon\}$ – the language containing only the empty string;
(c) $\{a\}$, for every symbol $a \in \Sigma$ – the language containing just that one single-character string.

Inductive cases. If L and M are regular languages, then each of the following is *also* a regular language:

(1) $L \cup M$ – their union;
(2) LM – their concatenation;
(3) L^* – the Kleene closure of L.

Closure clause. Nothing else is a regular language: the regular languages are exactly the languages reachable by finitely many applications of the three inductive cases above, starting only from the three base cases.

This definition tells us *which* languages count as regular. It does *not*, by itself, give us a convenient way to write such a language down on paper or in code – and that gap is exactly what a regular expression fills.

Regular Language vs. Regular Expression

A **regular language** is a set of strings satisfying the definition above – a semantic object, a mathematical fact about which strings belong to it. A **regular expression** (next lecture) is a piece of *syntax*: a compact, human-writable notation for naming one specific regular language. Every regular expression denotes exactly one regular language, and — a real theorem, proved properly in your Automata Theory course — every regular language can be named by at least one regular expression. The two notions therefore describe exactly the same class of languages, but they are conceptually different: one is a fact about a language, the other is a way of writing it down.

Exam Focus

"Is this a regular language?" and "is this a valid regular expression?" are different questions, and exam wording sometimes blurs them on purpose. A language can be regular even if you haven't yet written down a regular expression for it (you just haven't found the notation yet); conversely, once we define regular expressions formally next lecture, every string of regex syntax that type-checks will denote *some* regular language, whether or not that language is a useful one.

Key Takeaways

- A **lexeme** is matched source text; a **token** is the abstract `<name, attribute>` pair a lexer emits for it; a **pattern** is the rule connecting the two.
- Genuine **lexical errors** are things no pattern can match at all (illegal characters, unterminated strings/comments, malformed numbers) – a misspelled keyword is *not* one of these, since it's still a valid identifier lexeme.
- **Panic mode** reports an error, then discards characters one at a time until scanning can safely resume – simple, always terminates, but can occasionally cascade into extra reported errors from one root mistake.
- The big picture: **regular expression** $\rightarrow$ **NFA** $\rightarrow$ **DFA** $\rightarrow$ **minimized DFA** $\rightarrow$ **transition table** $\rightarrow$ **lexer**, via Thompson's construction, subset construction, and minimization (or a direct regex-to-DFA shortcut) – every step mechanical, which is exactly why lexer generators can automate all of it.
- The **Chomsky hierarchy** nests formal languages by how much machine power they need: regular (DFA = NFA) $\subset$ context-free (PDA, strictly more powerful nondeterministic than deterministic) $\subset$ context-sensitive (LBA) $\subset$ recursively enumerable (Turing machine, the most powerful model known).
- A **regular language** is a set of strings, defined inductively from three base cases ($\emptyset$, $\{\varepsilon\}$, $\{a\}$) and three inductive cases (union, concatenation, Kleene closure). A **regular expression** – next lecture – is the notation we use to name one.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 4

Regular Expressions, Regular Definitions, and Extensions; Regular vs. Non-Regular Languages; Real-World Lexer Design and Flex

CLO(s) covered:	CLO 1
Dragon Book [DB]:	§3.3 (Specification of Tokens: regular expressions, regular definitions, extensions)
Other references:	[PP] Programming Language Pragmatics §3.3

Lecture Roadmap

1. **Regular expressions**: the formal, inductive notation for naming the regular languages defined last lecture, plus many worked examples.
2. **Regular definitions**: naming sub-expressions, culminating in DB's classic identifier and numeric-literal definitions.
3. **Extensions** (`+`, `?`, character classes) – convenience, not new power – and **regular vs. non-regular languages**: what a lexer's patterns fundamentally cannot express.
4. How real-world lexers extend this model, and how the regular expressions in this lecture map directly onto the syntax used by **Flex**, the lexer-generator tool this course uses.

Lecture 3 defined the regular *languages* themselves, as sets of strings. Today gives us the *notation* – regular expressions – for writing them down, and by the end of this lecture you will be able to read and write the formal notation a real lexer specification is built from.

4.1 Regular Expressions

[DB] A **regular expression** is a compact notation for describing a language. DB defines regular expressions *inductively*: a small set of base cases, plus rules for combining smaller regular expressions into bigger ones – deliberately mirroring, symbol for symbol, the definition of the regular languages themselves from §3.7.

Definition: Regular Expressions (Inductive Definition)

Over alphabet Σ:

- **Base case 1:** ε is a regular expression; $L(\varepsilon) = \{\varepsilon\}$.
- **Base case 2:** for each symbol $a \in \Sigma$, a is a regular expression; $L(a) = \{a\}$.
- **Inductive case (union):** if r and s are regular expressions, so is $r \mid s$, with $L(r \mid s) = L(r) \cup L(s)$.
- **Inductive case (concatenation):** if r and s are regular expressions, so is rs, with $L(rs) = L(r)L(s)$.
- **Inductive case (closure):** if r is a regular expression, so is r^*, with $L(r^*) = (L(r))^*$.
- **Parenthesization:** if r is a regular expression, so is (r), with $L((r)) = L(r)$ – used purely to control grouping/precedence.

Here $L(r)$ denotes "the regular language named by regular expression r" – notice every base case

and inductive case above produces a regular language in exactly the sense of §3.7, and nothing else, matching that definition rule for rule.

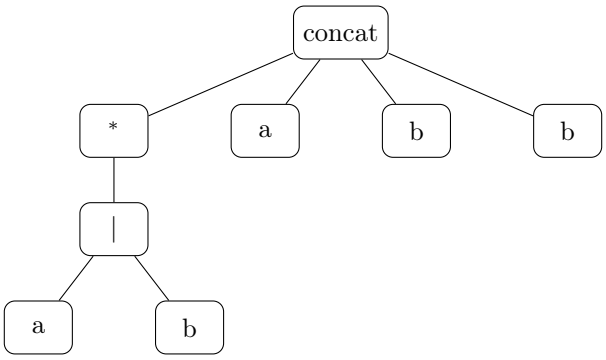
Operator Precedence

When parentheses are omitted, DB fixes a precedence, highest to lowest: **closure** (*) binds tightest, then **concatenation**, then **union** (|) binds loosest. So $ab \mid c$ means $(ab) \mid c$, *not* $a(b \mid c)$; and $a \mid bc^*$ means $a \mid (b(c^*))$.

4.1.1 Worked Example: Building Up a Regular Expression

DB's running example: the language of all strings of `a`'s and `b`'s that *end in* `abb`. Let's build the regular expression piece by piece.

Sub-expression	What it describes
$a \mid b$	a single `a` *or* a single `b`
$(a \mid b)^*$	*any* string of `a`'s and `b`'s (including ε)
$(a \mid b)^*abb$	any string of `a`'s and `b`'s, **followed by** the literal suffix `abb` – i.e. exactly the strings that *end in* `abb`



Syntax tree for $(a \mid b)^*abb$: the root concatenates four pieces — the starred union, then three literal symbols.

Example: Trace: Does `"babb"` Match?

`babb` = `b` (matched by the $(a|b)^*$ part, one repetition) followed by `a`, `b`, `b` (the literal suffix). Yes, it matches. Does `"abab"` match? No – it does not end in the literal substring `abb` (it ends in `bab`).

Exam Focus

Given an informal language description ("strings that start with …", "strings with an even number of …", "strings that don't contain …"), be able to construct the regular expression, and vice versa: given a regular expression, describe its language in English and list 2–3 example strings it does and does not match. This two-way translation is the most common exam pattern for this topic.

4.1.2 More Worked Examples

Language (in English)	Regular Expression	Matches / Doesn't Match
Binary strings with an even number of 0's	$1^*(01^*01^*)^*$	Matches `110011`; doesn't match `100`
Strings over $\{a, b\}$ containing at least one `a`	$(a \mid b)^* a (a \mid b)^*$	Matches `bba`; doesn't match `bbb`
Strings over $\{a, b\}$ *not* containing the substring `aa`	$b^*(ab^+)^*a^?$	Matches `ababb`; doesn't match `baab`
C-style single-line comment (using Flex's `.` for "any character except newline"[1])	`"//" .*`	Matches `// TODO fix this`; doesn't match a two-line comment

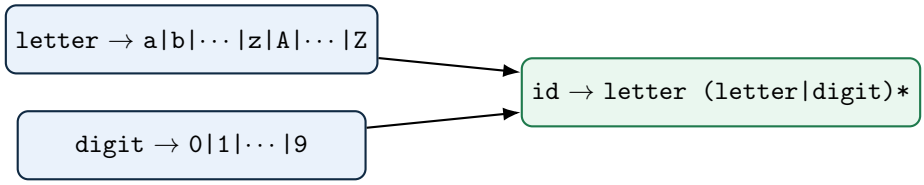
4.2 Regular Definitions

[DB] Writing out a complex regular expression as one giant unreadable formula is painful. A **regular definition** lets us name intermediate regular expressions and reuse those names in later definitions — exactly like defining helper variables in ordinary algebra.

Definition: Regular Definition

A regular definition is a sequence $d_1 \to r_1,\ d_2 \to r_2,\ \ldots,\ d_n \to r_n$ where each d_i is a new name, and each r_i is a regular expression over $\Sigma \cup \{d_1, \ldots, d_{i-1}\}$ – that is, r_i may use the alphabet *and* any name defined *before* it, but never itself or a later name (no recursive or forward references).

4.2.1 Worked Example: Identifiers

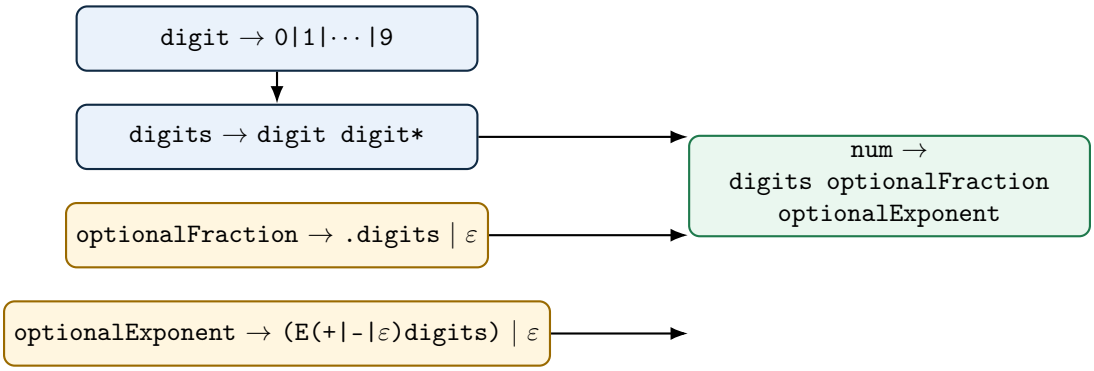


This is exactly $L(L \cup D)^*$ from the worked example in §3.7, now written as a named, reusable regular definition – and it is the pattern for token `ID` in almost every C-family language.

4.2.2 Worked Example: Unsigned Numbers (DB's Classic Definition)

Numbers like `5280`, `0.01`, `6.336E4`, or `1.89E-4` all need to be matched by a single token pattern. DB builds this up from four named pieces:

[1] Writing this with only the six core operators from §4.1 would require an enormous union of every symbol in Σ except newline. Real tools sidestep this with a dedicated wildcard; Flex's is `.`, and its full regex syntax – including this quoted-literal `"//"` convention – is covered in §4.6.5.



Example: Tracing `num` Against `6.336E4`

`digits` matches `6`; `optionalFraction` matches `.336` (the `.digits` branch); `optionalExponent` matches `E4` (the ε-sign branch of `(E(+|-|ε)digits)`, since no `+`/`-` appears). Concatenated: `6` + `.336` + `E4` = `6.336E4`. For a plain integer like `5280`, both `optionalFraction` and `optionalExponent` take their ε branch and contribute nothing.

Exam Focus

Be able to reproduce this exact four-piece `num` definition from memory, and trace it against a given number (integer, decimal, or scientific-notation) the way the example above does. This is DB's single most frequently examined regular-definition example.

4.3 Extensions of Regular Expressions

[beyond DB] Modern regex notation (and DB §3.3.5) adds a few extra operators. Formally, none of these add any expressive power beyond plain union/concatenation/closure — they are **syntactic sugar**, purely for convenience and readability.

Extension	Meaning	Equivalent to Core Regex
r^+	one or more repetitions	rr^*
$r^?$	zero or one occurrence (optional)	$r \mid \varepsilon$
$[abc]$	character class: any one of `a`, `b`, `c`	$a \mid b \mid c$
$[a\text{-}z]$	character range: any letter from `a` to `z`	$a \mid b \mid \cdots \mid z$
$[\hat{}a]$	negated class: any symbol *except* `a`	union of all $x \in \Sigma \setminus \{a\}$

Exam Focus

Know that $r^+ \equiv rr^*$ and $r^? \equiv (r \mid \varepsilon)$ *exactly* — a common trick question asks you to rewrite an extended-notation regex using only the three core operators (union, concatenation, closure), which is precisely this substitution.

4.4 Regular vs. Non-Regular Languages

[DB] Not every language can be described by a regular expression. This matters enormously for compiler design: it is exactly *why* a single lexer pass is not enough, and why we need an entirely different, more powerful formalism (context-free grammars, covered once the course reaches parsing) for parsing.

The Central Limitation: No Counting

A regular expression (equivalently, a finite automaton, covered once the course reaches finite automata) has no way to *count* an unbounded quantity and later check that count against something else. It can verify local, bounded patterns, but it cannot verify *global balance*. This single limitation explains every classic example of a non-regular language.

Language	Regular?	Why
Identifiers: letter followed by letters/digits	**Yes**	Purely local: check each character against a fixed small set of rules, no counting needed.
Fixed-format numeric literals	**Yes**	Same reason – bounded structure, no unbounded counting or matching required.
Balanced parentheses, e.g. $\{a^n b^n \mid n \geq 0\}$	**No**	Requires counting how many `(` were seen and demanding *exactly* that many `)` later — an unbounded count with no fixed upper limit.
Matching HTML/XML open and close tags	**No**	Same reason, at the level of tags instead of characters – and worse, tags can *nest*, requiring a stack, not just a counter.
Palindromes over $\{a, b\}$	**No**	Requires comparing the first half of the string against the reverse of the second half – again, unbounded memory of what came before.

Example: Why Finite Automata Can't Count: A Concrete Illustration

Suppose you tried to build a finite automaton for balanced parentheses up to nesting depth n. You would need at least one distinct state per depth level, $0, 1, 2, \ldots, n$, just to remember "how many unmatched `(` have I seen so far." Since parentheses in real programs can nest arbitrarily deep, no *fixed, finite* number of states suffices for *every* possible program – yet a finite automaton is required to have a fixed number of states, decided in advance. This is the intuitive heart of the **pumping lemma for regular languages** from your Automata Theory prerequisite: pump the middle of a long enough matched string and you break the balance, proving no finite automaton can recognize it.

Bridging Idea – Why This Matters for the Rest of the Course

- Regular expressions/finite automata are the right tool for tokens: identifiers, numbers, keywords, operators – all locally-checkable, bounded patterns.
- Nested and recursive structure – matched parentheses, balanced braces, nested expressions, `if`/`else` matching – is fundamentally *beyond* what regular expressions can express, no matter how cleverly you write them.
- This is precisely why the course introduces **context-free grammars** once parsing begins: a strictly more powerful formalism (using a stack, conceptually) that *can* count and match nested structure, and precisely why the compiler needs *two* separate formalisms — one per phase — instead of one universal one.

4.5 Real-World Lexer Design

[beyond DB] Everything so far in this lecture follows DB's simplified model fairly closely. Production lexers extend that model in several practical directions worth knowing about, even though we will not build any of them by hand in this course.

4.5.1 Handling Keywords: The Reserved-Word Trick

A naive lexer might need a separate hand-written check for every keyword (`if`, `while`, `return`, ...) before falling back to the generic identifier pattern. Production lexers instead use a much simpler trick: pre-load the symbol table with every reserved word *before* scanning begins, each marked with its own keyword token. The lexer then has only *one* pattern for anything "letter followed by letters/digits." When it matches such a lexeme, it looks the lexeme up in the symbol table: if an entry already exists (because it's a reserved word), the lexer returns *that* token (e.g. `WHILE`); if no entry exists, it's a genuine user identifier, so the lexer creates a new symbol-table entry and returns `ID`. One pattern, one lookup, no special-casing in the automaton itself.

4.5.2 The Maximal Munch Rule

When several patterns could all match a prefix of the remaining input, real lexers apply the **longest-match** ("maximal munch") rule: always take the *longest* lexeme that matches *any* pattern. This is why `<=` is scanned as a single `RELOP` token and not as `<` followed by `=`, and why `i` in `int` is not prematurely cut off as an identifier before the lexer has looked far enough ahead to see the rest of the word. C's `--` (decrement) versus `- -` (two unary minuses) and `->` (member access through a pointer) are further classic examples where getting the lookahead right matters. We formalize *why* this rule is needed, and how a scanner implements it efficiently, once we cover finite automata in the lectures ahead.

4.5.3 Numeric and String Literals in Practice

DB's `NUM` pattern is deliberately simplified. Real languages support numeric literal formats far beyond plain decimal integers, and each variant is a distinct lexical pattern the scanner must recognize:

Format	Example
Hexadecimal / octal / binary	`0x1F`, `0o17`, `0b1010`
Scientific notation	`6.022e23`, `1.5E-10`
Digit-group separators	`1_000_000` (Python, Java, Rust, C++14)
Type suffixes	`3.14f` (float), `100L` (long), `42u` (unsigned)
Complex/imaginary literals	`3+4j` (Python)

Each of these is its own regular pattern (or family of patterns) that a real scanner's specification must include – one more reason lexer-generator tools like Flex (§4.6.5) are preferred over hand-written code once a language's literal syntax grows this rich.

String-literal lexing hides similar complexity. **Escape sequences** (`\n`, `\t`, `\\`, `\uXXXX`) must be recognized *inside* the literal's pattern, not treated as ending it. **Raw strings** (Python's `r"..."`, C#'s `@"..."`) deliberately *disable* escape processing, so a single string-literal token category can need two different sub-patterns. Hardest of all is **string interpolation** / **f-strings** (Python's `f"Total = {score}"`, JavaScript template literals): the lexer must recognize where an embedded *expression* begins inside a string and, in effect, briefly hand control back to a full expression lexer/parser before resuming the string literal. This breaks the assumption that lexical analysis is a single flat pass over characters, and is a genuinely active area of parser/lexer-design discussion in modern language implementations.

4.5.4 Beyond ASCII and Flat Text: Unicode and Indentation

DB's "a letter followed by letters and digits" quietly assumes ASCII. Modern language standards (Python 3, Swift, Julia, Rust as of recent editions) permit *any* Unicode character in the "Letter"

category as part of an identifier – so accented Latin letters (`café`), non-Latin scripts (Greek, Arabic, Devanagari, Han characters), and even emoji are all valid identifiers in the languages that permit them (Swift famously allows an emoji as a variable name). Real lexer specifications reference the Unicode standard's character-category tables rather than a simple `[A-Za-z]` range.

Not every language is purely character-driven in the first place. Python and Haskell use indentation, not braces, to delimit blocks. Their lexers cannot rely on simple per-character regular patterns alone – they maintain an explicit **indentation stack** and synthesize special `INDENT` and `DEDENT` pseudo-tokens whenever a logical line's indentation increases or decreases relative to the stack's top, in addition to the ordinary tokens for the line's actual content. This is a clean example of *why* DB's model of "patterns matched against a flat character stream" is the right *starting* point but not the end of the story for every real language.

4.5.5 Seeing Real Tokenizers at Work

You do not have to take any of this on faith – most language toolchains expose their tokenizer directly. Python's `python3 -m tokenize myfile.py` prints every token, its exact source position, and its type. Clang's `clang -Xclang -dump-tokens -fsyntax-only file.c` dumps the full token stream Clang's own lexer produced. Node.js/Acorn and Babel offer their own `-tokens` flags for JavaScript. Running one of these on a file with a deliberate typo – an unterminated string, an unexpected character – is the fastest way to see exactly what a real lexer reports as an error versus what it silently accepts and passes on to the parser.

4.6 Regex in Practice

[beyond DB] The formal regular-expression notation from this lecture is the shared theoretical core behind every "regex" you have ever typed into a search box, a script, or a lexer specification – but real-world tools diverge from that core, and from each other, in important ways.

4.6.1 Not All "Regex" Is Actually Regular

This is a genuinely important and often-missed subtlety. Tools like Perl, Python's `re`, and JavaScript's regex support **backreferences** – e.g. `(\w+) \1` matches any word immediately repeated, like `hello hello`. This is provably *not* a regular language in the formal sense used in this lecture — recognizing it requires comparing two substrings for equality, which needs unbounded memory, just like the palindrome example in §4.4. Backreference-supporting engines use **backtracking search**, not finite automata, under the hood; they are more accurately called "backtracking pattern matchers" than true regular-expression engines. Lexer-generator tools like Flex, by contrast, implement only *true* regular expressions, compiled to genuine finite automata, precisely because lexical analysis needs the speed and predictability guarantees only real finite automata provide.

4.6.2 Catastrophic Backtracking (ReDoS)

Backtracking regex engines can, on adversarial input, take *exponential* time on a pattern that looks entirely innocent. The classic example is `(a+)+b` applied to a long string of `a`'s with no trailing `b`, like `"aaaaaaaaaaaaaaaaaaaaaaaaa!"`: the engine tries exponentially many ways of splitting the a's between the inner and outer `+` before giving up. This is a real, named security vulnerability class — **ReDoS** (Regular expression Denial of Service) — and has caused real production outages (Cloudflare's July 2019 global outage was triggered by exactly this kind of catastrophic backtracking in a WAF rule). The illustrative growth pattern:

Input length (a's, no trailing b)	20	25	30	35
Illustrative matching steps (order of magnitude)	$\sim 10^5$	$\sim 10^6$	$\sim 10^8$	$\sim 10^{10}$

This is exactly why DFA-based matching – which is *provably* linear in input length, with no possibility of this blow-up – is not just a theoretical nicety but the actual engineering reason production lexers

never use a naive backtracking matcher.

4.6.3 Modern Guaranteed-Linear Engines

Google's **RE2** library and Rust's `regex` crate deliberately give up backreferences and a handful of other backtracking-only features specifically so that they can guarantee linear-time matching via automata-based execution, immune to ReDoS by construction. This is the industrial embodiment of the regular-vs-non-regular distinction from §4.4: by refusing to support genuinely non-regular features, these libraries buy back the performance guarantee that true finite automata provide.

4.6.4 Brzozowski Derivatives – An Alternative to Automata

The standard way to build a matcher for a regex is to first convert it to a finite automaton (via Thompson's construction, previewed in §3.5 and covered fully in the lectures ahead). There is a completely different, more recent technique worth knowing about: the **derivative** of a regular expression r with respect to a symbol a, written $\partial_a(r)$, is a new regular expression describing "what still needs to match after consuming an a." Repeatedly taking derivatives, one input symbol at a time, and checking whether the final derivative accepts ε, matches a string *without ever building an explicit automaton at all.* Janusz Brzozowski introduced this in 1964; it fell out of fashion for decades, then saw a resurgence starting around 2010 in functional-programming circles (Scala's and Racket's parser-combinator libraries use derivative-based matching), because derivatives compose beautifully with recursive data types in a way that hand-built automata do not.

4.6.5 Regular Expression Syntax in Flex

Flex, the lexer-generator tool this course uses, takes a set of regular expressions – one per token – and runs the *entire* pipeline from §3.5 for you, emitting a ready-to-compile C scanner. Its regex syntax is, deliberately, close to the formal core taught in this lecture, plus a handful of practical extensions:

Flex syntax	**Meaning**	**This lecture's notation**
`r1r2`	concatenation	$r_1 r_2$
`r1|r2`	union / alternation	$r_1 \mid r_2$
`r*`	Kleene closure	r^*
`r+`	one or more	r^+
`r?`	optional	$r^?$
`(r)`	grouping	(r)
`.`	any character except newline	shorthand for $\Sigma \setminus \{\text{newline}\}$
`[abc]`, `[a-z]`	character class / range	union of the listed symbols
`[^abc]`	negated class	union of $\Sigma \setminus \{a, b, c\}$
`{n}`, `{n,m}`, `{n,}`	exactly n (or n to m, or n or more) repetitions	r concatenated with itself, that many times
`"..."`	literal string – special characters inside need no escaping	concatenation of literal symbols
`\`	escape one special character	the literal symbol itself
`{name}`	reference a named regular definition from the Definitions section	exactly d_i in a regular definition (§4.2)
`^`, `$`	anchor: only at start / end of a line	a position assertion, not part of the core theory
`r1/r2`	trailing context: match r_1 only if followed by r_2 (lookahead, not consumed)	not part of the core theory

Example: The `num` Regular Definition, Written as Flex Would See It

§4.2.2's four-piece `num` definition translates almost verbatim into a Flex specification's Definitions section:

```
digit       [0-9]
digits      {digit}+
optFrac     (\.{digits})?
optExp      (E[+-]?{digits})?
num         {digits}{optFrac}{optExp}
```

Every named piece (`digit`, `digits`, `optFrac`, `optExp`, `num`) is exactly the d_i from the formal regular definition, just written with Flex's concrete `{name}` reference syntax instead of the abstract $d_i \to r_i$ notation – the underlying regular expression is identical either way.

Exam Focus

Two Flex features above – `{n,m}` interval counts and `r1/r2` trailing context – are *not* part of the six-rule inductive definition from §4.1. Be able to explain `{n,m}` as pure syntactic sugar (exactly like r^+ or $r^?$: it expands to n to m literal copies of r concatenated), the same way §4.3 treats the other extensions – but trailing context genuinely changes *which part of the match is consumed*, which is a new capability, not just notation.

A Bit of History

The regex you type into `grep`, Python, or a Flex specification are not identical languages – they share the core ideas from this lecture but differ in surface syntax and sometimes in power. POSIX ERE (`grep -E`) stays closest to DB's formal core, with no backreferences and character classes like `[:digit:]`. PCRE, Python's `re`, and JavaScript add backreferences and lookahead/lookbehind assertions – genuinely more expressive than regular languages, as §4.6 discussed. Knowing which flavor you're targeting matters: a pattern copied from a Python script into a Flex `.l` file may silently mean something different, or fail to compile at all.

4.7 Self-Check Questions

Practice – Not Graded

1. For the statement `total = total + rate * 60;`, write out the full stream of `<token-name, attribute-value>` pairs, following the style of Worked Examples 2 and 3.
2. Explain why `2x` (digit immediately followed by a letter) can be treated as a genuine lexical error in many languages, while `fi` cannot, even though both look like "obvious mistakes" to a human reader.
3. A lexer encounters an opening `"` with no matching closing quote before the end of the file. Describe two different, reasonable ways a lexer could recover from this and keep scanning the rest of the file.
4. Why must the longest-match (maximal munch) rule apply when scanning `i <= 10`, so that `<=` is not mistakenly split into `<` followed by `=`?
5. Explain, in your own words, why Python's lexer cannot be built from character-level patterns alone the way a C lexer can.

6. In the big-picture pipeline (§3.5), which single step is responsible for turning *ambiguity* (several possible next states) into a single, unambiguous next state? Name the algorithm.
7. Explain, without using the word "notation," the difference between a regular *language* and a regular *expression*.
8. Write a regular expression for all strings over $\{0, 1\}$ that *start* with `1`. Then write one for strings that *do not contain* the substring `00`.
9. Using DB's `num` regular definition from §4.2.2, trace the match for the literal `1.89E-4`, showing what each of `digits`, `optionalFraction`, and `optionalExponent` matches.
10. Rewrite r^+ and $r^?$ using only union, concatenation, and Kleene closure (no extension operators allowed).
11. Explain, in your own words and without using the word "count," why balanced parentheses of arbitrary depth cannot be recognized by any regular expression.
12. Translate the regular definition `id -> letter (letter|digit)*` into the `{name}`-based syntax a Flex Definitions section would use, matching §4.6.5's worked example.
13. A teammate writes the regex `(a|aa)+b` and complains their program hangs on long inputs with no trailing `b`. What phenomenon is this, and why does it happen with a backtracking engine but not with a DFA-based one?

Key Takeaways

- A **token** is an abstract `<name, attribute>` pair; a **pattern** is the rule describing which lexemes belong to a token; a **lexeme** is the actual matched source text. Keep these three cleanly separated.
- Lexer and parser are split into separate phases for **simplicity, efficiency, and portability** – know all three reasons.
- The lexer and parser typically interact via a pull-based `getNextToken()` call, not by materializing a full token list up front.
- Token attributes – almost always a symbol-table pointer for `ID` – are how later phases recover exactly *which* identifier or literal a token represents.
- Most apparent "typos" (like `fi` for `if`) are *not* lexical errors, because they are valid lexemes for some other token; genuine lexical errors are things no pattern can match at all (illegal characters, unterminated strings/comments, malformed numbers).
- The big picture: **regular expression → NFA → DFA → minimized DFA → transition table → lexer**, via Thompson's construction, subset construction, and minimization – every step mechanical, which is exactly why lexer generators can automate all of it.
- A **regular language** is a set of strings, defined inductively from $\emptyset$, $\{\varepsilon\}$, and $\{a\}$, closed under union, concatenation, and Kleene closure. A **regular expression** is the notation we use to name one – different concepts, same underlying class of languages.
- **Regular definitions** let you name and reuse sub-expressions; DB's `id` and `num` definitions are the two canonical worked examples to know cold.
- Extension operators (+, ?, character classes) add convenience, not expressive power – each has an exact equivalent using only the three core operators.
- Regular expressions fundamentally **cannot count** an unbounded quantity, which is exactly why balanced parentheses, nested tags, and palindromes are *not* regular – and exactly why the compiler needs a second, more powerful formalism (context-free grammars) for parsing.
- Not everything called "regex" in practice is regular in the formal sense – backreference-supporting engines use backtracking and can suffer catastrophic (ReDoS) performance; true

finite-automaton-based matching, which we build next, is immune to this by construction.

- Real-world lexers go well beyond DB's simplified patterns: the reserved-word trick, maximal munch, rich numeric/string literal formats, indentation-based tokens, and Unicode identifiers are all standard in production compilers – and Flex's own regex syntax (§4.6.5) maps almost directly onto everything formalized in this lecture.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

[PP] Scott, M. L. *Programming Language Pragmatics*. 4th ed. Morgan Kaufmann.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 5

Finite Automata: Deterministic and Nondeterministic; Thompson's Construction (Regular Expression → NFA)

CLO(s) covered:	CLO 1, CLO 2
Dragon Book [DB]:	§3.6 (Recognition of Tokens), §3.7 (Finite Automata; From a Regular Expression to an Automaton), Alg. 3.23 (Thompson's Construction)
Other references:	[AP] Modern Compiler Implementation, §2.3–2.4; [HMU] Introduction to Automata Theory, Languages, and Computation, Ch. 2

Lecture Roadmap

1. Why a regular expression alone isn't something a computer can *run* – and what we need instead.
2. **Finite automata**, formally: the 5-tuple definition, transition diagrams, and what it means for a string to be "accepted."
3. **DFA vs. NFA**: exactly one choice per step vs. several (or none) – and why Lecture 3's Chomsky-hierarchy table already told you these have *equal* power.
4. **Thompson's construction**: a purely mechanical algorithm that builds an NFA directly from the structure of a regular expression, one small piece per operator – worked all the way through on the running example from Lecture 4, $(a \mid b)^*abb$.

This lecture builds the second arrow in Lecture 3's big-picture diagram: regular expression → NFA. The next lecture builds the rest: NFA → DFA → minimized DFA → an actual running lexer.

5.1 Why We Need an Actual Machine

[DB] A regular expression is a piece of notation – it describes a language on paper, but a computer cannot "run" a piece of algebra directly. To actually scan a source file character by character and decide, in real time, which token pattern (if any) the input matches, we need something a processor can execute: a small, fixed set of states, and a rule for how to move between them as each character arrives. That is exactly what a **finite automaton** is, and Lecture 3's big-picture diagram (§3.5) already named the first mechanical step that gets us there: **Thompson's construction**, which converts any regular expression into an equivalent finite automaton. This lecture builds that step in full; the next lecture finishes the journey to a working, table-driven lexer.

5.2 Finite Automata, Formally

[DB] Before comparing deterministic and nondeterministic versions, it helps to pin down what a finite automaton actually *is* – the same formal-definition instinct §3.7 used for regular languages themselves.

Definition: Finite Automaton

A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where:

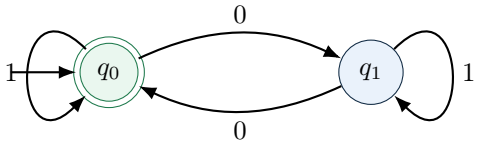
- Q is a finite set of **states**;

- Σ is the input **alphabet** (the same notion from §3.6);
- δ is the **transition function**, describing how to move between states as input symbols arrive (its exact type differs for DFAs and NFAs – see below);
- $q_0 \in Q$ is the **start state**;
- $F \subseteq Q$ is the set of **accepting** (or *final*) states.

A **transition diagram** is simply this 5-tuple drawn as a picture: a circle per state, a double circle for each accepting state, an arrow (labeled with an input symbol) per transition, and an unlabeled incoming arrow marking the start state – exactly matching the "transition diagram vs. transition table" distinction from §3.5's exam-focus box: same underlying automaton, two different ways of writing it down.

Example: Worked Example: Even Number of 0's

Recall from §4.1.2 that $1^*(01^*01^*)^*$ describes binary strings with an even number of `0`'s. Here is a two-state automaton for exactly that language: $Q = \{q_0, q_1\}$, $\Sigma = \{0, 1\}$, start state q_0, accepting states $F = \{q_0\}$ (start *and* accept – zero `0`'s is even).



Reading a `0` always *flips* which state you're in (tracking odd/even `0`'s seen so far); reading a `1` never changes the count, so it's a self-loop either way. Try tracing `1010`: start at q_0, read `1` (stay q_0), read `0` (move q_1), read `1` (stay q_1), read `0` (move q_0) – ends at $q_0 \in F$, accepted, matching that `1010` does indeed have an even number (two) of `0`'s.

What "Accepts" Means, Formally

Extend δ to $\hat{\delta}$, a function on *strings* rather than single symbols, by $\hat{\delta}(q, \varepsilon) = q$ and $\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$ for a string x followed by one more symbol a – in plain words, "follow the transitions one symbol at a time, starting from q." A string w is **accepted** by the automaton if $\hat{\delta}(q_0, w) \in F$: start at the start state, follow w symbol by symbol, and end up somewhere in the accepting set. The **language of the automaton**, written $L(\text{automaton})$, is the set of all strings it accepts – and for every automaton we build in this course, $L(\text{automaton})$ will turn out to be exactly the regular language we started with.

5.3 DFA vs. NFA: Two Models, Equal Power

[DB] The automaton above had exactly one arrow leaving each state for each symbol – no ambiguity about where to go next. That property has a name.

Definition: Deterministic Finite Automaton (DFA)

A finite automaton is **deterministic** if δ is a genuine function $Q \times \Sigma \to Q$: for every state and every input symbol, there is *exactly one* transition, and no transitions are labeled with ε. At every step, a DFA's next move is completely determined – hence the name.

Real regular expressions, though, are built from *union* – "match r *or* s" – which is naturally ambiguous: from a given state, on a given symbol, there might genuinely be more than one place

a matcher could go. Rather than resolve that ambiguity immediately, it is far more convenient (as Thompson's construction, §5.4, will show) to allow it explicitly in the automaton itself.

Definition: Nondeterministic Finite Automaton (NFA)

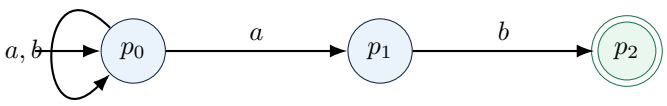
A finite automaton is **nondeterministic** if δ is a function $Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ (the power set of Q): for a given state and input symbol (or ε, the empty string), there can be **zero, one, or many** next states. Two new abilities appear here that a DFA does not have:

- **Multiple choices.** From one state, on one symbol, the automaton may be able to move to several different states at once.
- **ε-transitions.** The automaton may move between states *without reading any input at all* – these are the glue Thompson's construction uses to wire smaller automata together.

A string w is accepted by an NFA if *at least one* sequence of choices, following the symbols of w (and optionally taking any number of ε-transitions in between), ends in an accepting state – not every path has to succeed, just one.

Example: A Small NFA: Strings Ending in `ab`

$Q = \{p_0, p_1, p_2\}$, $\Sigma = \{a, b\}$, start p_0, accept $F = \{p_2\}$:



From p_0, on an `a`, there are *two* valid moves: stay at p_0 (self-loop) or advance to p_1 – exactly the nondeterminism a DFA cannot express directly. To accept `aab`: stay at p_0 reading the first `a` (via the self-loop), then move $p_0 \to p_1$ reading the second `a`, then $p_1 \to p_2$ reading `b` – one successful path is all it takes, even though *guessing wrong* (always taking the self-loop) would never reach p_2.

Exam Focus

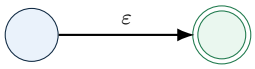
This "one successful path is enough" rule is exactly why simulating an NFA directly is awkward: naively, you'd have to try every possible sequence of choices to know whether *any* of them accepts. Lecture 6's subset construction sidesteps this entirely, by tracking *every reachable state at once* as a single set – turning "does some path accept?" into an ordinary DFA question. That NFA-to-DFA conversion is also the constructive proof that NFAs and DFAs recognize exactly the same class of languages (Lecture 3's Chomsky-hierarchy table already told you this fact; Lecture 6 shows *why* it's true).

5.4 Thompson's Construction: Regular Expression → NFA

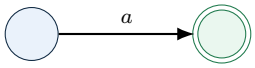
[DB] Thompson's construction (DB Algorithm 3.23) builds an NFA from a regular expression by *structural recursion* – exactly mirroring the six-rule inductive definition of a regular expression itself from §4.1. Every rule below produces a fragment with exactly **one** start state and exactly **one** accept state, which is what makes the rules compose so cleanly: a bigger fragment is always built by wiring together smaller fragments' single start/accept points, never by reaching inside them.

5.4.1 The Five Construction Rules

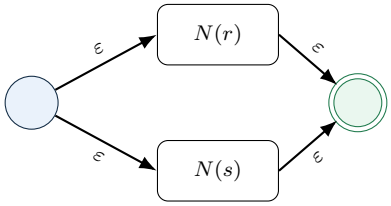
Base case 1: ε. A two-state fragment connected by a single ε-edge.



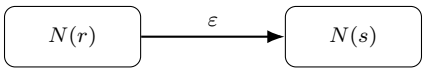
Base case 2: a single symbol $a \in \Sigma$. A two-state fragment connected by one a-edge.



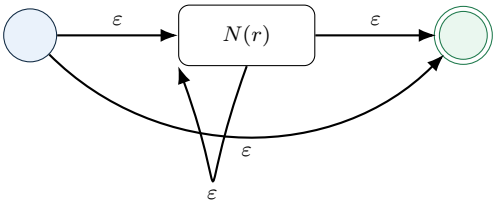
Inductive case: union $r \mid s$. Given fragments $N(r)$ and $N(s)$ (each with its own start/accept), build a new start and new accept state, and wire in *both* old fragments with ε-edges. The old accept states stop being accepting – only the new one is.



Inductive case: concatenation rs. Given $N(r)$ and $N(s)$, connect $N(r)$'s accept state to $N(s)$'s start state with a single ε-edge. $N(r)$'s start becomes the overall start; $N(s)$'s accept becomes the overall accept.



Inductive case: closure r^*. Given $N(r)$, build a new start and new accept state. Wire the new start directly to the new accept (ε, for zero repetitions) and to $N(r)$'s start (ε, to begin a repetition); wire $N(r)$'s accept back to $N(r)$'s start (ε, to loop again) and forward to the new accept (ε, to stop).



Exam Focus

Every fragment above has exactly **one** start and **one** accept state – even after combining fragments. This invariant is what makes the recursion work at all: at every step, you always know exactly which single state to wire the *next* ε-edge into, no matter how deeply nested the regular expression is. Losing track of this (e.g. trying to wire a new edge into "one of the accept states" of a union fragment) is the most common mistake when doing this construction by hand.

5.4.2 Worked Example: Building the NFA for $(a \mid b)^*abb$

[DB] This is exactly the regular expression §4.1.1 built up piece by piece with a syntax tree. We now build its NFA the same way, stage by stage, applying one construction rule per stage.

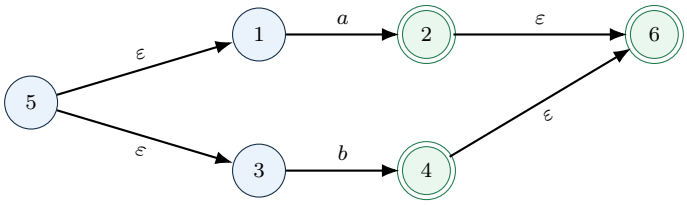
Example: Stage 1 – The Atoms

Build the two-state fragments for the individual symbols first. We'll need four of them: one for `a` inside the $(a \mid b)$, one for `b` inside the $(a \mid b)$, and one more each for the trailing `a`, `b`, `b`.

(States 1–2 recognize just `a`; states 3–4 recognize just `b`.)

Example: Stage 2 – Union: $(a \mid b)$

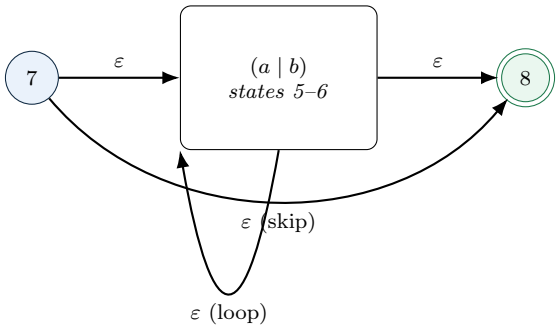
Apply the union rule to the two fragments above: new start state 5, new accept state 6.



(States 2 and 4 are no longer accepting – only the new state 6 is.)

Example: Stage 3 – Closure: $(a \mid b)^*$

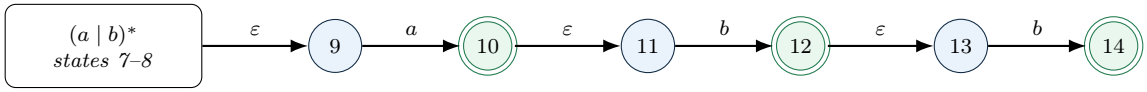
Apply the closure rule to the union fragment above (states 5–6): new start state 7, new accept state 8.



State 7 is now the start of everything built so far; state 8 is the accept state for $(a \mid b)^*$.

Example: Stages 4–6 – Concatenating a, b, b

Three more single-symbol fragments (states 9–10 for `a`, 11–12 for `b`, 13–14 for `b`), concatenated on one at a time: state 8 connects to state 9, state 10 connects to state 11, state 12 connects to state 13 – each via a single ε-edge, exactly the concatenation rule.



The complete NFA for $(a \mid b)^*abb$ has 14 states, start state 7, and a single accept state, 14. Every edge in every stage above is still present – concatenation only ever *adds* an edge, it never removes or renames anything from the fragments it connects.

Example: Tracing `"babb"` Through the Finished NFA

Recall from §4.1.1 that `babb` should be accepted. One successful path: $7 \xrightarrow{\varepsilon} 5 \xrightarrow{\varepsilon} 3 \xrightarrow{b} 4 \xrightarrow{\varepsilon} 6 \xrightarrow{\varepsilon} 8 \xrightarrow{\varepsilon} 9 \xrightarrow{a} 10 \xrightarrow{\varepsilon} 11 \xrightarrow{b} 12 \xrightarrow{\varepsilon} 13 \xrightarrow{b} 14$. Reading off just the symbols consumed along this path – b, a, b, b – reproduces `babb` exactly, and the path ends at state 14, the accept state. One successful path is all an NFA needs.

5.5 Self-Check Questions

Practice – Not Graded

1. Write out the formal 5-tuple $(Q, \Sigma, \delta, q_0, F)$ for the "even number of 0's" automaton in §5.2, giving δ as an explicit table (one row per state, one column per symbol).
2. Using $\hat{\delta}$ from the coreidea box in §5.2, trace whether the string `0110` is accepted by the same automaton. Show each intermediate state.
3. Explain, without using the word "choice," what it means for an NFA's transition function to have type $Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ rather than $Q \times \Sigma \to Q$.
4. In the "strings ending in `ab`" NFA (§5.3), list *all* paths (successful or not) that consume the string `aab`, and confirm exactly one of them ends in the accepting state.
5. Apply Thompson's construction by hand to the regular expression ab^*: draw the fragment for `a`, the fragment for b^*, and the concatenation that joins them.
6. Why does every fragment in Thompson's construction have *exactly one* start state and *exactly one* accept state, even after several rounds of union, concatenation, and closure? What would go wrong with the concatenation rule if a fragment were allowed more than one accept state?
7. For the finished 14-state NFA in §5.4.2, find one string of length 5 that is accepted and one string of length 5 that is *not* accepted, and briefly justify each.

Key Takeaways

- A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$; a **transition diagram** is just its picture, and a **transition table** is the same information as an array – interchangeable representations, not different machines.
- A **DFA** has exactly one transition per state per symbol and no ε-edges; an **NFA** allows several transitions (or none) per state per symbol, plus ε-edges that move without consuming input.
- An NFA accepts a string if *at least one* sequence of choices reaches an accepting state – not every path has to succeed.
- **Thompson's construction** builds an NFA from a regular expression using five rules (ε, a single symbol, union, concatenation, closure), each producing a fragment with exactly one start and one accept state – which is exactly what lets the rules compose recursively, mirroring the regular expression's own inductive definition rule for rule.
- The worked $(a \mid b)^*abb$ NFA (14 states) is the same running example from Lecture 4 – Lecture 6 converts this exact automaton into a DFA and then minimizes it, so it is worth understanding this construction solidly before moving on.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

[HMU] Hopcroft, J. E., Motwani, R., and Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Pearson, 2006.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 6

From NFA to a Working Scanner: Subset Construction, DFA Minimization, NFA vs. DFA Trade-offs, and the Design of a Lexical-Analyzer Generator

CLO(s) covered:	CLO 1, CLO 2, CLO 3
Dragon Book [DB]:	§3.7 (Alg. 3.20, Subset Construction), §3.9 (Alg. 3.36, DFA Minimization); §3.2 (Input Buffering), §3.5, §3.8 (Design of a Lexical-Analyzer Generator)
Other references:	[ET] Engineering a Compiler, §2.4–2.5; [AP] Modern Compiler Implementation, §2.5

Lecture Roadmap

1. **Subset construction** (Algorithm 3.20): the mechanical algorithm that turns any NFA into an equivalent DFA, worked all the way through on Lecture 5's 14-state NFA for $(a \mid b)^*abb$.
2. **DFA minimization** (Algorithm 3.36): merging behaviorally-identical states to produce the smallest possible DFA for the same language – applied to the very DFA we just built.
3. **NFA vs. DFA, revisited**: same recognizing power (Lecture 5 already proved this constructively), but a real space/time trade-off in practice – worth understanding concretely, not just as a slogan.
4. Zooming out from theory to engineering: **how a lexical-analyzer generator is actually built** – combining many token patterns into one automaton, implementing maximal munch by simulating a DFA, resolving pattern ties by rule order, and buffering the input stream efficiently. This is the generic architecture that *any* scanner-generator tool (Flex included) implements under the hood.

This lecture completes the pipeline Lecture 3's big picture (§3.5) promised: regular expression → NFA (done in Lecture 5) → DFA → minimized DFA → transition table → an actual, running lexer (all done here). By the end of today you will have followed one regular expression all the way from formal notation to working scanner code, and you will understand exactly what a tool like Flex is doing on your behalf when you hand it a `.l` file – the concrete Flex syntax itself is covered separately in the supplementary Flex notes, not here.

6.1 From an NFA to a DFA: The Subset Construction

[DB] Lecture 5 left us with a working NFA, but simulating an NFA directly is awkward: at every step there can be several possible next states, so a naive simulator would have to explore every possible sequence of choices to know whether *any* of them accepts (§5.3's exam-focus box already flagged this). The **subset construction** sidesteps the problem entirely with one elegant idea: instead of asking "which *one* state am I in?", track the entire *set* of NFA states that are reachable so far, all at once. A set of NFA states is itself just one state of a new automaton – and moving between these sets, one input symbol at a time, turns out to be completely deterministic.

6.1.1 Two Helper Operations

Definition: ε-closure and `move`

Let T be a set of NFA states.

- ε-closure(T) is the set of all NFA states reachable from *any* state in T using **zero or more** ε-transitions alone – "every state you could drift into for free, without reading any input." Every state in T is trivially in its own ε-closure (zero transitions is allowed).
- move(T, a) is the set of NFA states reachable by taking **exactly one** transition labeled $a \in \Sigma$ from *some* state in T – "every state you could be in immediately after consuming one a, before taking any further ε-transitions."

Every DFA state the subset construction builds is, underneath, one of these ε-closure sets – which is exactly why DFA states are conventionally drawn or labeled as *sets* of NFA state numbers throughout this section.

Algorithm 3.20 – Subset Construction

1. **Start state.** Compute $D_0 = \varepsilon\text{-closure}(\{n_0\})$, where n_0 is the NFA's start state. D_0 becomes the DFA's start state, and is added to a worklist of DFA states still needing their outgoing transitions computed.
2. **Process the worklist.** While the worklist is nonempty, remove one unmarked DFA state T from it. For *every* input symbol $a \in \Sigma$:
   (a) Compute $U = \varepsilon\text{-closure}(\text{move}(T, a))$.
   (b) If U is empty, there is no transition on a from T – skip it (this becomes an implicit "error"/dead transition, §6.1.3).
   (c) If U is a *new* set not seen before, add it as a new DFA state and place it on the worklist.
   (d) Either way, record the transition $\delta_{\text{DFA}}(T, a) = U$.
3. **Accepting states.** A DFA state T (itself a set of NFA states) is accepting if and only if T contains *at least one* accepting NFA state – one successful NFA path reaching an accept state is exactly what "the set contains an accepting state" captures.
4. **Termination.** Since the NFA has only finitely many states, it has only finitely many *subsets* of states ($2^{|Q|}$ of them, at most) – so this process, which only ever creates DFA states out of such subsets, is guaranteed to terminate; it cannot generate new DFA states forever.

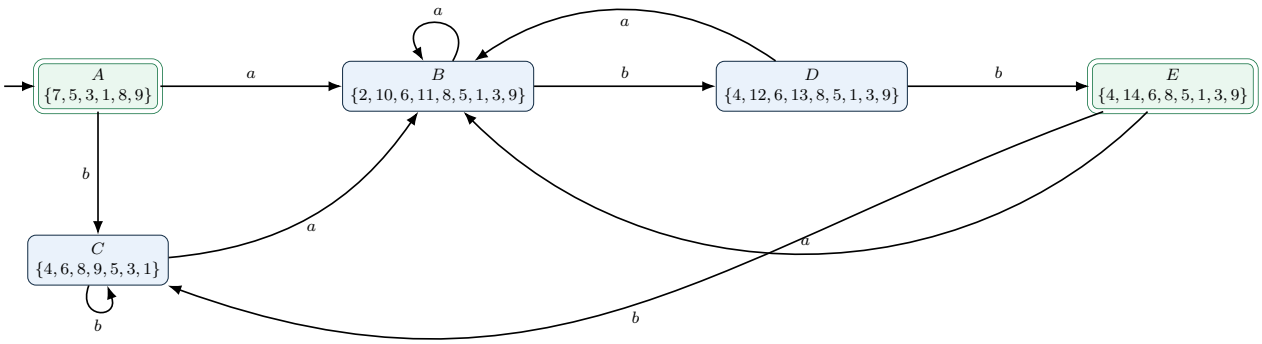
Exam Focus

The single most common hand-execution mistake: forgetting to take the ε-closure *after* `move`, not just at the very start. Every new DFA state is ε-closure(move(T, a)), *not* just move(T, a) – if you skip the closure step, you will silently drop reachable NFA states and build an incorrect DFA that rejects strings it should accept.

6.1.2 Worked Example: Building the DFA for $(a \mid b)^*abb$

[DB] We continue directly from Lecture 5's 14-state NFA (§5.4.2): start state 7, single accept state 14, and the ε- and symbol-transitions built stage by stage in that lecture. Applying subset construction:

DFA State	Set of NFA States (after ε-closure)	On a	On b
A (start)	$\{7, 5, 3, 1, 8, 9\}$ – everything reachable from 7 with no input at all	B	C
B	ε-closure(move$(A, a)) = \{2, 10, 6, 11, 8, 5, 1, 3, 9\}$	B	D
C	ε-closure(move$(A, b)) = \{4, 6, 8, 9, 5, 3, 1\}$	B	C
D	ε-closure(move$(B, b)) = \{4, 12, 6, 13, 8, 5, 1, 3, 9\}$	B	E
E (accepting)	ε-closure(move$(D, b)) = \{4, 14, 6, 8, 5, 1, 3, 9\}$	B	C



Example: Reading This DFA in Plain English

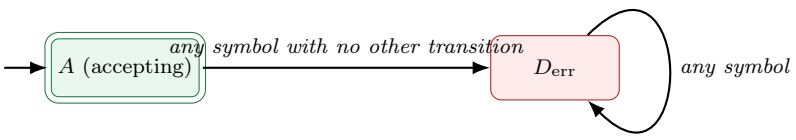
Only state E is accepting, and E is reached only by seeing an a then two b's in a row ($D \xrightarrow{b} E$, and D is reached only via $B \xrightarrow{b} D$, and B is reached only by having just read an a) – in other words, exactly "the last three characters read were `abb`." This is a direct, mechanical confirmation that the DFA really does recognize $(a \mid b)^*abb$: no hand-waving, just five states and the two helper operations above, run to completion.

Exam Focus

Notice the NFA had **14 states** but the DFA has only **5**. This is *not* a general rule – subset construction can just as easily *blow up* the state count (§6.3 works through why) – it simply happened to shrink here because many of this NFA's 14 states were ε-only "glue" states that collapse into the same reachable set. Do not assume the DFA is always smaller; always run the algorithm.

6.1.3 Dead States and the Implicit Error Transition

[DB] Step 2(b) of the algorithm above quietly skips any ε-closure(move(T, a)) that comes out empty. In a real scanner, "no valid transition on this symbol" cannot simply be ignored – it is exactly how a lexical error, or the end of the current token, gets detected. The standard way to make this explicit is to add one extra, non-accepting **dead state** D_{err} with a self-loop on every symbol, and route every missing transition to it.



Reaching D_{err} means the automaton is certain no continuation can ever lead back to an accepting state – so a real scanner treats arrival at D_{err} as the signal to stop extending the current match immediately, without even reading further input (there is no reason to; every path from D_{err} dead-ends). §6.4.2 shows exactly how this plugs into maximal-munch matching.

6.2 DFA Minimization

[DB] Subset construction guarantees a DFA, but not the *smallest* DFA for the language – different DFA states can behave identically in every possible future, making some of them redundant. **DFA minimization** (DB Algorithm 3.36) finds and merges exactly these redundant states, producing the unique smallest DFA (up to renaming states) recognizing the same language.

Definition: Distinguishable States

Two DFA states p and q are **distinguishable** if there exists *some* input string w such that following w from p ends in an accepting state while following w from q does not (or vice versa) – i.e. some future input reveals a difference in behavior between them. States that are *not* distinguishable by *any* string are **indistinguishable**, and can be safely merged into a single state without changing which strings the automaton accepts.

Algorithm 3.36 – Partition Refinement

Minimization works by repeatedly *refining* a partition of the states into groups that are *provisionally* treated as indistinguishable, splitting a group apart the moment a symbol is found that sends its members to genuinely different groups.

1. **Initial partition.** Split all states into exactly two groups: the accepting states F, and the non-accepting states $Q \setminus F$ (including the dead state, if one was added). This is the coarsest split that could possibly be correct, since an accepting and a non-accepting state are always distinguishable by the empty string $w = \varepsilon$ alone.
2. **Refine, repeatedly.** For each current group G and each input symbol a, check where each state in G goes on a. If every state in G lands in the *same* other group on every symbol, G stays together. If some states in G land in a *different* group than others (on some symbol a), split G into sub-groups accordingly – states now provably distinguishable (via that one symbol a) must never end up merged.
3. **Repeat until stable.** Keep refining until one full pass over every group and every symbol produces *no* further splits – at that point the partition has stabilized: every two states remaining in the same group really are indistinguishable by *any* string, not just the symbols checked so far.
4. **Build the minimized DFA.** Each final group becomes one state of the minimized DFA. The start state is the group containing the original start state; a group is accepting if it contains (any, and therefore every) original accepting state; transitions are inherited directly, since every state in a stable group agrees on where each symbol leads.
5. **Drop the dead state.** If minimization merges everything unreachable/hopeless into the dead-state group, that group is simply omitted from the final diagram – a missing transition in the minimized DFA is understood to mean "go to the (implicit) dead state and reject," exactly as §6.1.3 already described.

Exam Focus

The refinement in step 2 must be applied *simultaneously* across every symbol before deciding a group is stable – a group only survives a round if it agrees on *every* symbol in Σ, not just one. A common hand-execution error is stopping as soon as one symbol doesn't split a group, without checking the rest of Σ too.

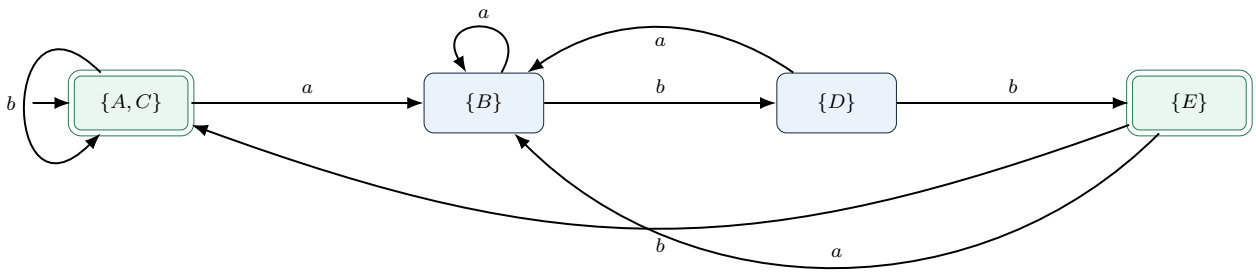
6.2.1 Worked Example: Minimizing the $(a \mid b)^*abb$ DFA

[DB] Apply the algorithm to the 5-state DFA built in §6.1.2 (A, B, C, D, E; only E accepting):

Round	Partition
Initial	$G_1 = \{A, B, C, D\}$ (non-accepting), $G_2 = \{E\}$ (accepting)
Round 1	Check G_1's members on each symbol. On a: $A \to B$, $B \to B$, $C \to B$, $D \to B$ – all land in G_1 (specifically all in B's group so far, fine). On b: $A \to C \in G_1$, $B \to D \in G_1$, $C \to C \in G_1$, but $D \to E \in G_2$ – **different group!** D must split off: G_1 becomes $G_{1a} = \{A, B, C\}$ and $G_{1b} = \{D\}$.
Round 2	Re-check $G_{1a} = \{A, B, C\}$: on a, all three go to $B \in G_{1a}$ – consistent. On b: $A \to C \in G_{1a}$, $B \to D \in G_{1b}$, $C \to C \in G_{1a}$ – B **disagrees!** Split again: G_{1a} becomes $\{A, C\}$ and $\{B\}$.
Round 3	Re-check $\{A, C\}$: on a, both go to B's group – consistent. On b: $A \to C$, $C \to C$ – both land in the *same* group $\{A, C\}$ itself – consistent. No further split needed for this group. All other groups are singletons, so they are trivially stable.
Stable	$\{A, C\}$, $\{B\}$, $\{D\}$, $\{E\}$ – four groups, no further splits possible.

Example: Why A and C Merge

Intuitively: A ("seen nothing useful yet, or just finished a full `abb` and are starting over") and C ("just saw a `b` that isn't continuing an `ab` prefix") behave *identically* from here on – from both, an a starts a fresh attempt at matching `abb`, and a b leaves you in exactly the same "waiting" situation. No future string can ever tell A and C apart, which is precisely the formal definition of indistinguishable states being applied.



The minimized DFA: 4 states instead of 5 – A and C have collapsed into one state, and every transition is inherited unchanged (e.g. $E \xrightarrow{b} C$ becomes $E \xrightarrow{b} \{A, C\}$). This is provably the smallest possible *DFA for $(a \mid b)^*abb$: DB Algorithm 3.36 guarantees minimality, not just a smaller result.*

Exam Focus

"Smallest possible" is a real, provable guarantee, not just "smaller than before." Two different minimized DFAs for the *same* language are always identical up to renaming states – there is exactly one minimal DFA (isomorphism aside), which is why minimization is sometimes described

as computing a *canonical form* for a regular language.

6.3 NFA vs. DFA, Revisited: Power and Performance

[DB] Lecture 5 and §3.6.1 already established that NFAs and DFAs recognize *exactly* the same class of languages – the subset construction above is the constructive proof of that fact, run by hand. Equal recognizing power does *not* mean equal practical cost, though, and this distinction is worth making concrete now that you have built both an NFA and a DFA for the same language.

	NFA (simulated directly)	**DFA (pre-built, table-driven)**
States tracked per input symbol	A *set* of up to $\lvert Q_{\text{NFA}}\rvert$ states must be tracked and updated at every step (recomputing an ε-closure each time)	Exactly **one** state at all times – a single array index
Work per input symbol	$O(\lvert Q_{\text{NFA}}\rvert)$ or worse, since every state in the current set may need its own transitions and closure explored	$O(1)$ – one table lookup, `table[state][symbol]`
Number of automaton states	Exactly as many as Thompson's construction produced – linear in the size of the regular expression	Can be **exponentially larger** ($2^{\lvert Q_{\text{NFA}}\rvert}$ in the worst case) – see the classic blow-up example below
When the cost is paid	Every single run, on every input, pays the per-symbol simulation overhead	Paid **once**, offline, when the DFA (and its table) is built; every subsequent run is just table lookups

Example: The Classic Exponential Blow-up

Consider the regular expression $(a \mid b)^* a (a \mid b)^n$ for some fixed n – "the n-th-from-last character is an `a`." Its NFA (via Thompson's construction) has only $O(n)$ states. But its *minimal* DFA needs 2^n states: intuitively, the DFA must remember the *last n characters seen* (to know, once n more characters arrive, whether that remembered character really was n-from-last), and there are 2^n possible values for "the last n characters," each requiring a genuinely distinguishable state. This is a real, provable worst case, not a contrived edge case – it is exactly why some lexer generators offer a choice between DFA-based and NFA-based matching for pathological patterns.

Why Lexer Generators Overwhelmingly Choose the DFA Side

For *ordinary* lexical tokens (identifiers, numbers, keywords, operators), the exponential worst case above essentially never triggers in practice – real token patterns are small and well-behaved, so their DFAs stay modest in size. Given that, the DFA's $O(1)$-per-character, pre-paid-once cost is a clear win for a component like a lexer that runs on *every single character* of every source file, potentially billions of times across a large codebase. This is precisely why every mainstream lexer generator (Flex included) compiles down to a DFA rather than simulating an NFA at scan time, even though building that DFA (subset construction, then minimization) is itself extra

one-time work at generator-build time.

Exam Focus

Contrast this with §4.6's discussion of backtracking regex engines (Perl, Python, JavaScript): those engines simulate something closer to an NFA *with backreferences* at match time, which is exactly why they can suffer ReDoS (§4.6) – unlike a lexer generator, they cannot always fall back to a pre-built DFA, because backreferences push the language being matched outside the regular languages entirely (§4.6 again). A lexer's patterns, by contrast, are always *genuinely* regular by construction, so the DFA route is always available.

Exam Focus

Lazy ("on-the-fly") DFA construction is the practical middle ground worth knowing by name: tools like Google's RE2 and many `grep` implementations build DFA states *on demand*, the first time they are actually reached during a real match, and cache them for reuse – getting the DFA's fast steady-state lookup without paying to build every theoretically-possible state up front. This matters most for patterns where the full DFA would be large but any single input only ever visits a small fraction of its states.

6.4 Designing a Lexical-Analyzer Generator

[DB] Everything so far in this lecture – and the two before it – has been about turning *one* regular expression into *one* DFA. A real lexer, though, must recognize *dozens* of different token patterns (`ID`, `NUM`, every keyword, every operator, ...) at once, decide which pattern produced the *longest* match at every position (maximal munch, §4.5), and hand the winning token off to the parser. This section builds that generic architecture – exactly what a lexer-generator tool constructs for you, and exactly what *any* such tool (Flex included) is doing under the hood, whatever its concrete input syntax looks like. *(The concrete `.l`-file syntax for Flex specifically is covered in the separate supplementary Flex notes, not here.)*

6.4.1 Combining Many Patterns Into One Automaton

[DB] A lexer-generator's input is fundamentally a list of **(pattern, action)** pairs – one regular expression per token, paired with the code that should run when that token is matched (typically: build the right `<token-name, attribute-value>` pair and return it). The combining step is a direct generalization of the union rule from Thompson's construction (§5.4.1):

1. Build (or imagine) a separate small NFA fragment $N(r_i)$ for each individual pattern $r_1, r_2, \ldots, r_k$, exactly as in Lecture 5.
2. Introduce one new overall start state, with an ε-transition to *every* fragment's own start state – precisely the union construction, generalized from two fragments to k.
3. **Critically, do *not* merge the fragments' accept states into one shared accept state** the way an ordinary two-way union would. Instead, each fragment $N(r_i)$ keeps its *own* accept state, tagged with *which* token r_i corresponds to.
4. Run subset construction (§6.1) on this combined NFA exactly as before. A resulting DFA state is accepting if its underlying set of NFA states contains *any* tagged accept state – and it is tagged with *that* token (or, if more than one tagged accept state appears in the same set, with whichever pattern is listed *first* in the original specification – see §6.4.3 below).

Example: A Miniature Combined Lexer: Two Patterns

Suppose a lexer needs only two token patterns: `KEYWORD_IF` for the literal string `if`, and `ID` for `letter(letter|digit)*`. Thompson's construction gives two small fragments; the combining step wires a shared start state into both, with each fragment's own accept state tagged `IF` or `ID` respectively. Subset construction then merges these into one DFA. Scanning the input `if5`: after `i`, then `f`, the DFA is in a state whose underlying NFA-state set includes *both* the `IF` fragment's accept state *and* the `ID` fragment's accept state simultaneously (since `if` is a valid lexeme for *both* patterns) – but reading one more character, `5`, only the `ID` fragment has anywhere left to go, so the DFA continues and eventually accepts `if5` as a single `ID` token, illustrating exactly why maximal munch (next subsection) has to keep reading past an early, shorter match.

6.4.2 Maximal Munch, Implemented as DFA Simulation

[DB] §4.5 introduced the longest-match rule informally. With a combined DFA in hand, it has a precise, mechanical implementation:

The Maximal-Munch Scanning Loop

1. Starting from the DFA's start state and the current input position, repeatedly follow transitions on successive input characters, **remembering the most recent point at which the current state was accepting** (and which token it was tagged with).
2. Keep advancing as long as a transition exists for the next character – *even past* an accepting state, since a longer match might still be possible (exactly the `if5` example above: the DFA is accepting for `IF` right after `if`, but a real scanner does not stop there).
3. Stop advancing the moment either (a) the input is exhausted, or (b) the next character has no valid transition from the current state – i.e. the only remaining move is to the implicit dead state D_{err} from §6.1.3. There is never a reason to enter D_{err} itself: once every remaining transition would lead there, the scan is over.
4. **Roll back** to the *last remembered accepting state* (step 1), emit a token for whichever pattern that state was tagged with, and resume scanning from the character right after that point.
5. If the current state was *never* accepting at any point during the scan (no remembered position exists at all), this is a genuine lexical error (§3.4): report it and invoke panic-mode recovery (§3.4.1).

Example: Tracing `<=` Against a Combined `RELOP` DFA

Suppose the DFA has transitions recognizing both `<` (alone, tagged `LT`) and `<=` (tagged `LE`). Scanning `<=10`: after `<`, the DFA is already in an accepting state (tagged `LT`) – remember this position. Reading `=` still has a valid transition (to the `LE`-tagged accepting state) – so keep going and update the remembered position to *here* instead (a strictly longer match was just found). Reading the next character, `1`, has *no* transition from the `LE` state – stop. Roll back to the last remembered position (right after `=`), emit `<LE, ->`, and resume scanning at `1`. This is exactly *why* `<=` is never mis-split into `<` followed by `=`, made completely mechanical.

6.4.3 Resolving Ties: What Happens When Two Patterns Match Equally Long

[DB] Maximal munch resolves *length* ties (always prefer the longer match), but a different kind of tie remains possible: two *different* patterns matching the *exact same* lexeme, of the exact same length. The reserved-word trick from §4.5 is one instance of this (`while` matches both a specific keyword pattern and the generic `ID` pattern) – and lexer generators resolve it with one simple, memorize-able rule:

Priority by Order of Declaration

When several patterns match a lexeme of the *same* maximal length, the lexer-generator convention is to prefer whichever pattern was **listed first** in the original specification. This is exactly why, in practice, keyword patterns like `if` and `while` are always written *before* the generic `ID` pattern in a lexer specification's rule list – listing them first guarantees the keyword-specific token wins the tie, without needing the reserved-word symbol-table trick from §4.5 at all (both techniques solve the same problem; a lexer generator typically uses rule ordering, while a hand-written lexer more often uses the symbol-table trick).

Exam Focus

Rule order controls only *same-length* ties. It never overrides maximal munch itself: a *longer* match from a pattern listed *later* in the file still wins over a *shorter* match from a pattern listed earlier. Get the two straight: **length** beats **position in the file**; position in the file is only the tie-breaker when lengths are *equal*.

6.4.4 Input Buffering: Reading Characters Efficiently

[DB] Maximal munch (§6.4.2) has one uncomfortable-sounding consequence: the scanner sometimes reads several characters *past* the eventual end of a token before it knows to roll back (the `<=` example needed to peek at `1` before committing to `<=`). A naïve implementation that requests one character at a time from the operating system, and that can only look one step ahead, would need awkward, slow special-casing to support this "read ahead, then roll back" behavior. DB's classic fix is a purely mechanical buffering scheme that makes rollback essentially free.

Definition: Double Buffering with Sentinels

Maintain *two* buffers of equal, fixed size (each, e.g., one disk block), laid out contiguously in memory, plus two pointers:

- `lexemeBegin` marks the start of the lexeme currently being matched;
- `forward` scans ahead, one character at a time, exactly as maximal munch's simulation loop (§6.4.2) advances the DFA.

When `forward` reaches the end of one buffer, the *other* buffer is refilled (one system call, one full block) while scanning continues seamlessly across the boundary. Once a token is recognized, `lexemeBegin` jumps forward to just past it, and the cycle repeats. Rolling back `forward` to a remembered accepting position (§6.4.2) is therefore just moving a pointer *backward within already-buffered memory* – no re-read from disk, no special-casing at all.

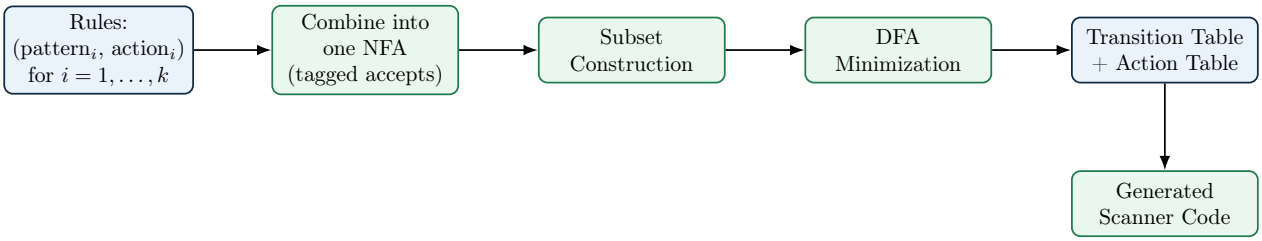
A further trick avoids testing "have I run off the end of the buffer?" on *every single* character read (which would otherwise double the cost of the scanner's innermost loop): place a special **sentinel** character (conventionally `eof`, distinct from every real input character) at the very end of *each* buffer. The scanner's inner loop then needs only *one* check per character – "did I just read the sentinel?" – and only in that comparatively rare case does it do the extra work of figuring out *which* buffer's sentinel was hit (real end of file) versus a routine buffer-swap point.

Exam Focus

Be able to explain *why* double buffering (rather than a single buffer, or reading one character at a time from the OS) is necessary specifically because of maximal munch's "read-ahead-then-roll-back" behavior – this input-buffering material is frequently tested as a short conceptual

question ("why can't a lexer just read one character at a time from disk?") rather than as a diagram-drawing exercise.

6.4.5 Putting It All Together: The Generic Lexer-Generator Pipeline



Everything left of "Generated Scanner Code" happens once, offline, when the lexer-generator tool runs on your rules file. What actually ships and runs on every source file afterward is only the rightmost box: a tight loop implementing §6.4.2's maximal-munch simulation over the pre-built table, driven by the double-buffered input of §6.4.4, and exposed to the parser via exactly the `getNextToken()` *interface §3.1 described. This is the complete answer to "what does a lexer-generator tool actually do" – Flex is one concrete implementation of this exact pipeline, and its own* `.l`*-file syntax for expressing the* Rules *box is covered separately.*

Closing the Loop Back to Lecture 3's Big Picture

Lay §3.5's original roadmap diagram next to this section's pipeline and they match exactly, with one addition: Lecture 3 promised *regular expression → NFA → DFA → minimized DFA → transition table → lexer*, for *one* pattern. This lecture generalizes every arrow in that promise to *many* patterns at once (via the tagged-union combining step) and fills in the one detail the original diagram left implicit – how "transition table → lexer" actually happens, via maximal-munch simulation over double-buffered input. Nothing new was invented to make this jump; the entire generalization is the union rule from Thompson's construction, applied k times instead of twice. What is still missing – and what Lecture 7 picks up next – is the *other* way to reach a working scanner: writing one by hand, diagram by diagram, with no generator tool at all, and weighing that choice honestly against everything built here.

6.5 Self-Check Questions

Practice – Not Graded

1. Apply subset construction by hand to the NFA for ab^* (built in Lecture 5's self-check question 5). List each DFA state as a set of NFA states, and draw the resulting DFA.
2. For the DFA you built in the previous question, apply DFA minimization. Are any states mergeable? Justify your answer in terms of distinguishability.
3. Explain, in your own words, why ε-closure must be applied *both* when computing the start state *and* again after every single `move`, not just once at the beginning.
4. Construct a small regular expression of your own (different from $(a \mid b)^*a(a \mid b)^n$) where you believe the DFA will have *more* states than the NFA, and explain your reasoning informally.
5. Trace the maximal-munch scanning loop (§6.4.2) on the input `intx` against a combined DFA for the two patterns `int` (keyword) and `letter(letter|digit)*` (`ID`), assuming keyword patterns are listed first. What single token is emitted, and why?

6. Two patterns, `a+` and `a+b?`, are both listed in a lexer specification, with `a+` listed first. For the input `aaa` (with no trailing `b` anywhere), which pattern's action fires, and is this a length tie or a maximal-munch decision?
7. Explain why a lexer cannot safely process input one character at a time, read directly from disk on every single character, given how maximal munch works – and explain concretely what a sentinel character saves the scanner's inner loop from having to do on every iteration.

Key Takeaways

- **Subset construction** converts any NFA into an equivalent DFA by tracking *sets* of NFA states as single DFA states, using ε-closure and `move` as its two building blocks; termination is guaranteed because a finite NFA has only finitely many state subsets.
- A **dead state** makes the "no valid transition" case explicit, and is exactly the signal maximal-munch scanning uses to know a match cannot be extended any further.
- **DFA minimization** (partition refinement) merges states that no future input string could ever tell apart, producing the provably smallest DFA for a language – a canonical form, unique up to state renaming.
- NFAs and DFAs recognize *exactly* the same languages, but the practical trade-off is real: DFA simulation is $O(1)$ per character with a pre-paid one-time build cost, while NFA simulation pays a per-character cost proportional to the current state set on *every* run – and a DFA can, in the worst case, be **exponentially larger** than its source NFA.
- A lexer-generator tool builds *one* combined DFA out of *many* token patterns by generalizing Thompson's union rule to k patterns, tagging each pattern's own accept state; **maximal munch** is then just a DFA-simulation loop that remembers the last accepting position seen and rolls back to it; **same-length ties** between patterns are broken by whichever pattern was listed first.
- **Double buffering with sentinels** is what makes maximal munch's read-ahead practical: rollback becomes a cheap pointer move within already-buffered memory, and the sentinel collapses the scanner's per-character bounds check into a single, rare-case test.
- This lecture completes Lecture 3's original big-picture promise in full: regular expression → NFA → DFA → minimized DFA → transition table → working, table-driven, maximal-munch lexer – every arrow now built, worked by hand, and tied back to a concrete implementation architecture. Lecture 7 now looks at the same destination from the opposite direction: building a scanner by hand, without a generator, and comparing the two approaches honestly.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.

THE NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, LAHORE

CS-4031 Compiler Construction — Fall 2026

Lecture 7

Transition Diagrams and Lexer Implementation Strategies: Ad Hoc vs. Table-Driven Scanning

CLO(s) covered:	CLO 1, CLO 3
Dragon Book [DB]:	§3.4 (Recognition of Tokens: transition diagrams for `relop`, identifiers/keywords, and whitespace; sketching a lexical analyzer directly from transition diagrams)
Other references:	[ET] Engineering a Compiler, §2.2–2.3 (hand-written scanners); [AP] Modern Compiler Implementation, §2.5

Lecture Roadmap

1. **Transition diagrams**: a lighter-weight, one-pattern-at-a-time cousin of the DFAs built in Lecture 6 – exactly the tool DB §3.4 uses to go from "here is a regular expression" to "here is code," worked through on `relop`, identifiers/keywords, and whitespace.
2. **Retraction**: what it means for a state to need to "look one character too far ahead," and how that connects directly back to Lecture 6's double-buffering scheme.
3. **The ad hoc approach**: translating a set of transition diagrams straight into hand-written scanning code, one routine per token category, the way many real compilers' lexers are actually built.
4. **Ad hoc vs. table-driven, head to head**: the two genuinely different ways to *implement* a lexer – hand-coded diagrams (today) vs. the generic, generated DFA simulation Lecture 6 already built in full – and when a real engineer reaches for each one.

Lectures 5–6 gave you the *formal* machinery (NFA, subset construction, minimization) that a lexer-generator tool runs automatically. This lecture asks a different, very practical question: what if you are writing the scanner *by hand*, without a generator at all? DB introduces transition diagrams for exactly this reason, and – not coincidentally – this is also where Project Phase 1 and Assignment 1 begin.

7.1 From Formal Automata to a Concrete Scanner

[DB] DB actually introduces transition diagrams (§3.4) *before* it formalizes NFAs and DFAs – historically, hand-drawn diagrams like these are how lexers were built for years before subset construction and minimization were automated into generator tools. Having already done Lectures 5–6 in full formal generality, we are in a good position to see transition diagrams for what they really are: **a deliberately simplified, by-hand DFA**, drawn separately for *one* token category at a time, skipping the machinery (§6.4.1's tagged-union combination, full subset construction) that a generator needs to merge *many* categories into a single automaton automatically.

Why Bother, Given Lecture 6 Already Solved This?

Lecture 6 answered "how does a lexer-generator tool build a scanner automatically." This lecture answers a different, equally real question: "what does a programmer actually *write*, by hand, when no generator tool is used at all?" Both questions matter – production compilers are built

both ways, sometimes even the *same* compiler mixing both strategies for different parts of its lexer (§7.6 returns to this). Understanding the hand-written route also makes the generated route feel less like magic: every diagram in this lecture is exactly the kind of small DFA fragment that subset construction (§6.1) would have produced for you, had you chosen to generate instead of hand-write it.

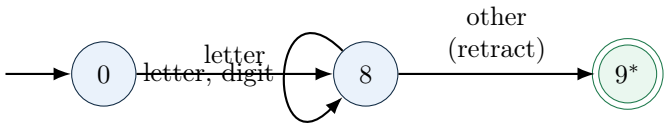
7.2 Transition Diagrams for Individual Token Patterns

Definition: Transition Diagram (DB §3.4 Sense)

A **transition diagram** here is a DFA-like picture built for *one* token category at a time: circles for states, labeled arrows for the character(s) that trigger each transition, and one or more accepting states. It differs from the general DFAs of Lecture 6 in one specific way: some accepting states are marked with a $*$, meaning "this state accepts, but only after the scanner has already read one character *too far* to know that – that extra character must be pushed back before the token is considered complete." This extra bookkeeping step is called **retraction**, covered in §7.3 below.

7.2.1 Worked Example: Identifiers and Keywords

[DB] The simplest and most-reused diagram: one letter, then any number of letters or digits – exactly Lecture 4's `id` regular definition (§4.2.1), just drawn as a picture instead of written as a formula.

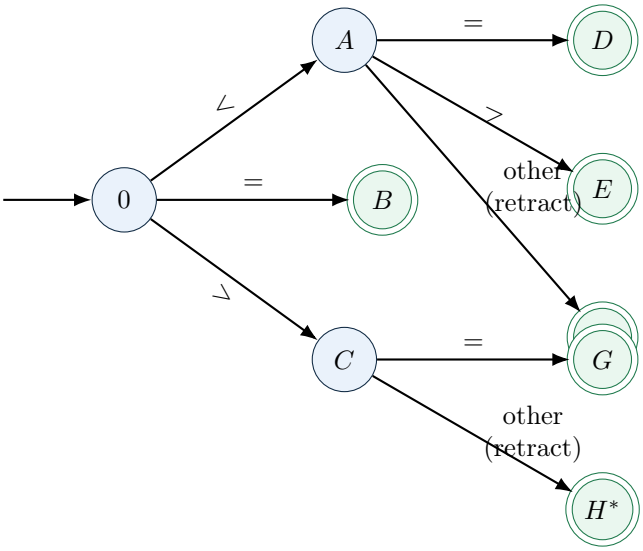


Example: Reading Diagram: Identifier/Keyword

State 8 self-loops on every letter or digit – exactly maximal munch, one character at a time. The moment a character arrives that is *neither* a letter nor a digit (a space, an operator, end of file), that character cannot be part of the identifier, so state 9 accepts – but state 9 is marked $*$, because the scanner already consumed that one extra, non-identifier character to *discover* the identifier was finished. §7.3 explains exactly what happens to that character next. Once the lexeme is known, the reserved-word trick from §4.5 decides `ID` vs. a specific keyword token via a symbol-table lookup – the diagram itself cannot tell `while` apart from `score`; only the lookup afterward can.

7.2.2 Worked Example: `relop` (DB's Classic Diagram)

[DB] DB's single most-cited transition-diagram example: the six relational operators `<`, `<=`, `<>`, `=`, `>`, `>=`. It is worth tracing by hand once, since it is the clearest possible illustration of *why* retraction is needed at all – some branches consume exactly one extra character to confirm a longer match, others don't.



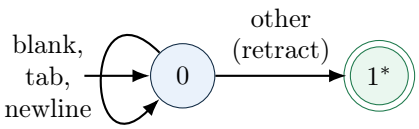
State	Token, Attribute	Retract Needed?
B	`RELOP`, EQ (=)	No – `=` alone is never a prefix of a longer `relop` in this language, so the very next character read is already known to belong to whatever comes *after* this token.
D	`RELOP`, LE (<=)	No – both characters of the lexeme were consumed and both were needed.
E	`RELOP`, NE (<>)	No – same reasoning as D.
F^*	`RELOP`, LT (<)	**Yes** – reaching F means the scanner read `<` *and then* some other character (to rule out `<=` and `<>`) that turned out not to belong to this token at all.
G	`RELOP`, GE (>=)	No.
H^*	`RELOP`, GT (>)	**Yes** – same reasoning as F.

Exam Focus

Notice *exactly* which states need the asterisk, and why: only the two states (F, H) reached by taking the "other" branch out of a two-character lookahead need retraction. States reached by consuming a character that was itself part of a *necessary*, meaningful transition (B, D, E, G) never need it. A very common exam mistake is marking every accepting state with a $*$ out of caution – retraction is needed *only* when a character was read purely to *rule out* a longer match and then turned out not to belong to the token at all.

7.2.3 Worked Example: Whitespace

[DB] One more diagram worth seeing explicitly, because it is the one diagram that produces *no token at all* – exactly §3.2's "housekeeping" point that whitespace and comments are consumed and discarded, not emitted as tokens.



Example: Why Whitespace Still Needs Retraction

State 0 self-loops through any run of spaces, tabs, or newlines. The instant a non-whitespace character appears, that is exactly the retract situation again: the scanner has read one character belonging to the *next* real token, purely to discover whitespace has ended. State 1* "accepts," but instead of emitting a token, the scanner simply **restarts** matching from state 0 of whichever diagram applies to the retracted character – whitespace is the one pattern in a real lexer specification whose successful match produces zero tokens, not one.

7.3 Retraction and the Buffer, Revisited

[DB] Every $*$ in the diagrams above means the same concrete thing: §6.4.2's maximal-munch loop already described this exact situation generically ("keep advancing... roll back to the last remembered accepting position"), and §6.4.4's double-buffering scheme is precisely what makes that rollback cheap. Nothing new is being invented here – transition diagrams just make the *single most common form* of that rollback (exactly one character of overshoot) visible and explicit, state by state, instead of leaving it implicit inside a generic simulation loop.

Retraction, Concretely

Reaching a $*$ state means: move the `forward` pointer (§6.4.4) back by exactly one position before recording where the lexeme ends. Because §6.4.4's buffers keep recently-read characters in memory, this is a pointer decrement, not a re-read from disk – exactly why double buffering was worth building in the first place. In hand-written, ad hoc code (§7.4 below), this is often implemented even more directly: use a **non-consuming lookahead** (a `peekChar()` that reads the next character *without* advancing the input pointer) instead of an advance-then-retract pair. The two are behaviorally identical – "peek, then only advance if the character turns out to belong to this token" never overshoots in the first place, so there is nothing left to retract.

7.4 From Diagrams to Code: The Ad Hoc Approach

[DB] With a transition diagram in hand for every token category, **ad hoc** lexer construction is exactly what the name suggests: translate each diagram directly into ordinary code, by hand, one routine (or one block of a larger routine) per category, dispatched by inspecting the first character of whatever remains in the input.

Definition: Ad Hoc Lexer

An **ad hoc lexer** is a hand-written scanner in which each transition diagram becomes an explicit block of code – typically an `if`/`switch` dispatch on the first character, followed by a small loop or a short chain of character tests that walks that one diagram's states directly, with no shared, generic "simulate a table" loop underneath. The programmer, not a generator tool, is responsible for combining the diagrams (deciding dispatch order, handling the reserved-word trick, wiring in retraction) correctly.

Example: A Small Ad Hoc Scanner, Sketched

The three diagrams from §7.2 translate almost line for line:

```
Token getNextToken() {
    skipWhitespace();          // Sec. 17-diagram-whitespace, inlined
    c = peekChar();            // non-consuming lookahead (Sec. 17-retraction)

    if (isLetter(c)) {         // Sec. 17-diagram-id
```

```
        lexemeBegin = forward;
        do { advance(); c = peekChar(); }
        while (isLetter(c) || isDigit(c));
        lexeme = getLexeme(lexemeBegin, forward);
        if (isKeyword(lexeme))
            return makeToken(keywordTokenFor(lexeme));
        return makeToken(ID, installID(lexeme));
    }

    if (c == '<') {            // Sec. 17-diagram-relop, the '<' branch
        advance();
        if (peekChar() == '=') { advance(); return makeToken(RELOP, LE); }
        if (peekChar() == '>') { advance(); return makeToken(RELOP, NE); }
        return makeToken(RELOP, LT);   // F*/H*: peek already IS the retraction
    }
    if (c == '=') { advance(); return makeToken(RELOP, EQ); }
    if (c == '>') {
        advance();
        if (peekChar() == '=') { advance(); return makeToken(RELOP, GE); }
        return makeToken(RELOP, GT);
    }

    // ... one block per remaining category (NUM, LITERAL, ...) ...

    reportLexicalError(c);
}
```

Notice the F^*/H^* states from §7.2.2 never actually call an explicit "retract" function here at all – using `peekChar()` throughout means the input pointer only ever moves forward past a character once that character is confirmed to belong to the current token, so there is nothing to undo. This is the standard real-world way ad hoc scanners sidestep explicit retraction bookkeeping, exactly as §7.3's coreidea box described.

Exam Focus

Be able to translate a small transition diagram (given to you on an exam) directly into pseudocode of this shape: one `if`/`switch` branch per outgoing edge from the start state, a loop for any self-loop, and a clear point where each accepting state either returns a token or (for whitespace) simply restarts the dispatch. This diagram-to-code translation is exactly what Assignment 1 and Project Phase 1 ask you to do by hand.

7.5 The Table-Driven Approach, Revisited

[DB] Lecture 6 already built this approach in full, so this section is a short recap, not new material: combine *every* token pattern's NFA into one, via the tagged-union construction (§6.4.1); run subset construction (§6.1) and minimization (§6.2) once, offline; encode the result as a transition table; and drive that table with *one generic loop* – §6.4.2's maximal-munch simulation – that has no idea, and does not need to know, how many token categories it is handling. A lexer-generator tool like Flex automates this entire pipeline (§6.4.5) from a rules file you write, and never once produces or reasons about a hand-drawn diagram like today's.

Exam Focus

Keep straight which lecture built *which* half: Lecture 6 built the **table-driven** half (one combined automaton, one generic simulation loop, machine-built via subset construction). *This* lecture built the **ad hoc** half (many small, separately hand-drawn diagrams, each translated into its own bespoke code). Both are complete, correct ways to end up with a working lexer – they differ in *how* the lexer is constructed, not in what it ultimately does.

7.6 Ad Hoc vs. Table-Driven: A Head-to-Head Comparison

	Ad Hoc (this lecture)	**Table-Driven (Lecture 6)**
How it's built	By hand: draw one diagram per token category, translate each into its own block of code	Automatically: combine all patterns' NFAs, run subset construction and minimization once
Combining many patterns	The programmer's responsibility – dispatch order, reserved-word handling, and retraction must all be gotten right by hand for *every* diagram	Handled uniformly by the tagged-union construction (§6.4.1) and rule-order tie-breaking (§6.4.3) – correct by construction
Adding a new token	Write (and correctly integrate) a new block of code, ordered correctly relative to every existing block	Add one line to the rules file and regenerate – the tool re-derives the whole combined automaton
Performance	Can be *very* fast – a human can special-case the hottest patterns (e.g. single-character operators) with no generic overhead at all	Also fast ($O(1)$ per character via table lookup, §6.3), but pays for whatever the combined table's size turns out to be
Correctness guarantees	None automatic – a forgotten retraction, a keyword checked after (not before) the generic `ID` branch, or a missed edge case is a real, easy-to-make bug	Strong – subset construction and minimization are provably correct algorithms; if the rules file is right, the generated scanner is right
Debuggability	Ordinary source code – step through it in any debugger, exactly like any other hand-written function	A generic loop over an opaque table – debugging usually means inspecting *table contents* and current state number, one level more abstract
Best fit	Small, fixed, performance-critical token sets where hand-tuning pays off, or where no generator tool is available/desired	Larger or evolving token sets, and any situation where the specification (the rules file) should be the single source of truth

Extra Depth: The Hybrid Reality

Just as Lecture 1 showed that "compiled vs. interpreted" is rarely a pure either/or choice in practice (most real systems are hybrids), the same is true here. CPython's own tokenizer and V8's (Chrome/Node.js JavaScript engine) lexer are both, in large part, **hand-written** C/C++ code – not output from Flex or a similar tool – precisely because a hand-tuned fast path for the handful of hottest patterns (identifiers, common operators, string-literal boundaries) can outperform a fully generic table-driven loop, while still calling out to smaller, more mechanically-generated pieces (e.g. Unicode category tables) for the parts where hand-tuning would not pay off. GCC's own lexer has, at different points in its history, used both strategies. The lesson: "ad hoc vs. table-driven" is a real engineering choice with real trade-offs (as the table above lays out), not a solved question with one universally correct answer – exactly why this course teaches both.

Where This Fits: Project Phase 1 and Assignment 1

This lecture is deliberately timed to sit right before you start writing code of your own. **Assignment 1** (regular expressions and finite automata) and **Project Phase 1** (the lexer stage of your semester project, first due around Lecture 7 in the master course outline) both draw directly on today's material: you will draw transition diagrams for your own language's tokens, and then choose – or be told to use – one of the two implementation strategies above to turn those diagrams into a running scanner, before Lecture 6's Opt. 1 hands-on session shows the alternative, generator-based route (Flex) side by side with what you just built by hand.

7.7 Self-Check Questions

Practice – Not Graded

1. Draw a transition diagram, in the style of §7.2.2, for a hypothetical `arrow` token family: `-`, `->`, `-` (decrement). Mark every state that needs retraction, and justify each marking.
2. For DB's `relop` diagram (§7.2.2), trace the input `<>=` character by character. Which token is emitted first, and what happens to the remaining `=`?
3. Explain, without saying "because DB says so," why state B (matching bare `=`) in §7.2.2 never needs retraction, while states F^* and H^* always do.
4. Translate the whitespace diagram (§7.2.3) into pseudocode in the same style as §7.4's worked example, and explain why it never calls `makeToken(...)`.
5. A classmate's ad hoc lexer checks the generic identifier pattern *before* checking for specific keywords like `while`. What token does their lexer incorrectly emit for the lexeme `while`, and how does §4.5's reserved-word trick fix this, concretely, in code?
6. Using §7.6's comparison table, argue *for* using an ad hoc lexer for a small teaching language with 15 fixed token types, and separately argue *against* using ad hoc for a language whose token set is still actively evolving (like an in-development scripting language). What changes between the two scenarios?
7. Explain concretely how using a non-consuming `peekChar()` (§7.3) avoids ever needing an explicit "move the pointer back one" operation, even though every $*$-marked state in this lecture's diagrams conceptually represents exactly that situation.

Key Takeaways

- A **transition diagram** is a small, by-hand DFA for *one* token category; states marked $*$ are exactly the states Lecture 6's maximal-munch loop reaches by reading one character past the true end of the token.
- **Retraction** means giving that one overshot character back, so scanning can resume from the right place – cheap in practice because of Lecture 6's double-buffered input, and often sidestepped entirely in hand-written code by using a non-consuming `peekChar()` lookahead instead.
- DB's classic `relop` diagram is the cleanest worked illustration of *when* retraction is and isn't needed: only branches that read a character purely to *rule out* a longer match, and then discover it doesn't belong, need it.
- The **ad hoc** approach translates each diagram directly into hand-written, bespoke code – fast and fully debuggable, but every bit of correctness (dispatch order, keyword handling, retraction) is the programmer's own responsibility.
- The **table-driven** approach (Lecture 6, in full) combines every pattern into one automaton mechanically and drives it with one generic, provably-correct simulation loop – easier to extend and to trust, at the cost of being one level more abstract to debug.
- Real production lexers (CPython, V8, historically GCC) often **mix both strategies**, hand-tuning the hottest patterns while generating or table-driving the rest – exactly the same "no single universally correct answer" lesson Lecture 1 already drew for compiled vs. interpreted execution.
- This lecture is the direct on-ramp to **Assignment 1** and **Project Phase 1**: drawing your own transition diagrams and choosing an implementation strategy for your own language's tokens.

References for This Lecture

References

[DB] Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D. *Compilers: Principles, Techniques, and Tools*. 2nd ed. Pearson, 2006.

[ET] Cooper, K. D. and Torczon, L. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.

[AP] Appel, A. W. *Modern Compiler Implementation in C/Java/ML*. Cambridge University Press, 2004.